
COP290 – Assignment 3

LevelDB: Making Changes in a Large Codebase

Section 1: Analysis of the Write Path and Compaction Mechanism

Section 2: Design and Implementation of the RangeScan API

Section 3: Design and Implementation of the DeleteRange API

Section 4: Design and Implementation of the Manual Full Compaction

Aditya Anand (2023CS50284) and Neelkanth Mishra (2023CS10022)

Indian Institute of Technology Delhi

April 2026

Contents

1	Analysis of the LevelDB Write Path, Read Path, and Compaction Mechanism	4
1.1	Introduction to LevelDB and the LSM-Tree	4
1.2	The Write Path	4
1.2.1	Stage 1 – Write-Group Formation (DBImpl::Write, db/db_impl.cc)	4
1.2.2	Stage 2 – Write-Ahead Log (db/log_writer.h, db/log_writer.cc)	5
1.2.3	Stage 3 – MemTable Insertion (db/memtable.cc, db/skiplist.h)	6
1.2.4	Stage 4 – MemTable Rotation (MakeRoomForWrite, db/db_impl.cc)	7
1.2.5	Stage 5 – Flushing to an SSTable (CompactMemTable, db/db_impl.cc)	8
1.3	The Read Path	9
1.3.1	Snapshot Pinning and Version Reference	9
1.3.2	Probe Order: MemTable, ImmMemTable, SSTables	9
1.3.3	Iterator-Based Range Reads (DBIter, db/db_iter.cc)	11
1.3.4	Read Amplification	12
1.4	The Compaction Mechanism	12
1.4.1	Triggering: Score-Based Selection (db/version_set.cc)	13
1.4.2	Input Selection and Overlap Expansion (db/version_set.cc)	14
1.4.3	The Merge Process (DoCompactionWork, db/db_impl.cc)	14
1.4.4	Trivial Move (db/db_impl.cc – BackgroundCompaction)	15
1.4.5	Memtable-Flush Priority	15
1.4.6	Write Amplification	16
1.5	Key Design Observations	16
2	Design and Implementation of the RangeScan API	17
2.1	Motivation	17
2.2	Public API Changes	17
2.2.1	include/leveldb/db.h	17
2.2.2	db/db_impl.h	17
2.3	Implementation – DBImpl::Scan (db/db_impl.cc)	18
2.3.1	Full-Compaction Guard	18
2.3.2	Core Scan Loop	18
2.4	Role of DBIter	19
2.4.1	Constructor Change – Pre-Fetching Range Tombstones	19
2.4.2	FindNextUserEntry – Three Suppression Filters	19
2.4.3	FindPrevUserEntry – Reverse Iteration	20
2.5	DBImpl::GetRangeDeletions (db/db_impl.cc)	20
2.6	Snapshot Isolation and Correctness	21
2.7	Complexity	21
3	Design and Implementation of the DeleteRange API	21
3.1	Motivation	21
3.2	New Value Type and Thread-Safe Structures – db/dbformat.h	22
3.2.1	Extended ValueType Enum	22
3.3	RangeDeletion and RangeDeletionList – db/dbformat.h and db/dbformat.cc	22
3.3.1	RangeDeletionList::Add (db/dbformat.cc)	23

3.3.2	RangeDeletionList::IsDeleted – MVCC Semantics and Comparator Injection (db/dbformat.cc)	23
3.3.3	RangeDeletionList::MergeInto (db/dbformat.cc)	24
3.3.4	RangeDeletionList::GetDeletions (db/dbformat.cc)	24
3.4	Public API Changes	24
3.4.1	include/leveldb/db.h	24
3.4.2	include/leveldb/write_batch.h	24
3.5	Write Path – WriteBatch Encoding (db/write_batch.cc)	25
3.5.1	WriteBatch::DeleteRange	25
3.5.2	WriteBatch::Iterate Dispatch	25
3.5.3	MemTableInserter::DeleteRange	25
3.6	MemTable Storage – db/memtable.h and db/memtable.cc	26
3.6.1	memtable.h Changes	26
3.6.2	MemTable::Add Hook (memtable.cc)	26
3.7	DBImpl::DeleteRange (db/db_impl.cc)	26
3.8	SSTable Persistent Path	27
3.8.1	include/leveldb/table.h Changes	27
3.8.2	Table::Rep and Rep Destructor (table/table.cc)	27
3.8.3	Table::ReadMeta Restructure (table/table.cc)	27
3.8.4	Table::ReadRangeDeletions (table/table.cc)	28
3.8.5	Table::GetRangeDeletions (table/table.cc)	28
3.8.6	Table::InternalGet Changes (table/table.cc)	28
3.9	TableBuilder Changes (table/table_builder.cc)	29
3.9.1	New range_del_block Member	29
3.9.2	TableBuilder::Add – Routing Range Tombstones	29
3.9.3	TableBuilder::Finish – Persisting the Range Deletion Block	30
3.10	TableCache Changes (db/table_cache.h and db/table_cache.cc)	30
3.10.1	table_cache.h	30
3.10.2	table_cache.cc – GetRangeDeletions	30
3.11	Version::Get Saver – db/version_set.cc	31
3.12	Compaction Integration – DoCompactionWork (db/db_impl.cc)	31
3.12.1	Collecting Input Range Tombstones	31
3.12.2	Sorting and Deduplicating Tombstones	31
3.12.3	Interleaving Tombstones with Point Keys	32
3.12.4	Dropping Point Keys Covered by Range Tombstones	32
3.12.5	Output File Boundary Tracking	32
3.12.6	Propagating Active Tombstones Across Output Files	33
3.13	Memtable Flush Boundary Fix (db/db_impl.cc – WriteLevel0Table)	33
3.14	db/write_batch_test.cc	33
3.15	db/version_set.h – Friend and Accessor	34
3.16	Interaction Between Range and Point Tombstones	34
3.17	Summary of Data Flow	34
4	Design and Implementation of the Manual Full Compaction Mechanism	34
4.1	Motivation and Overview	34
4.2	New Data Structures (db/db_impl.h)	35
4.2.1	SingleCompactionRecord	35
4.2.2	StatsForCompaction	35

4.2.3	New DBImpl Members	35
4.3	State Flag: force_full_compaction_in_progress_	36
4.3.1	Write Throttling in Write()	36
4.3.2	Throttling Get() and NewIterator()	36
4.3.3	Scheduler Bypass in MaybeScheduleCompaction()	36
4.3.4	Helper Methods	36
4.3.5	RecordForceCompactionStats	37
4.4	Destructor Change	37
4.5	FlushedMemTable() (db/db_impl.cc)	37
4.6	CompactWholeLevel() (db/db_impl.cc)	38
4.7	Execution Flow of ForceFullCompaction()	38
4.7.1	Phase 1 – Acquire Control and Initialise Records	38
4.7.2	Phase 2 – Flush the MemTable	39
4.7.3	Phase 3 – Wait for Existing Background Work	39
4.7.4	Phase 4 – Sequential Level Draining	39
4.7.5	Phase 5 – Cleanup and Broadcast (FinishFullCompaction lambda)	40
4.7.6	Phase 6 – Statistics Aggregation and Output	40
4.8	Write() Signal Broadcast	41
4.9	Interaction with the Background Compaction Thread	41
4.10	Correctness Considerations	42
5	Evaluation of Implemented Features	42
5.1	RangeScan Evaluation	43
5.2	DeleteRange Evaluation	43
5.3	Manual Full Compaction Evaluation	44
5.4	Explanation of Compaction Statistics Using a Simple Case	47
5.5	Summary of Results	49
6	Resources	50

Analysis of the LevelDB Write Path, Read Path, and Compaction Mechanism

Introduction to LevelDB and the LSM-Tree

LevelDB is a high-performance, persistent key-value storage library developed by Google. It exposes a minimal public interface — `Put(key, value)`, `Get(key)`, `Delete(key)` — yet internally implements a sophisticated storage engine based on the **Log-Structured Merge-Tree** (LSM-Tree).

The central idea of the LSM-Tree is that *sequential* disk I/O is orders of magnitude faster than *random* disk I/O. LevelDB exploits this by never modifying on-disk data in-place; every mutation becomes a sequential append in memory, and sorted in-memory batches (*SSTables*) are flushed and merged in the background.

The on-disk structure is a hierarchy of **levels** $L_0, L_1, \dots, L_6$. Each level's size target grows geometrically:

$$\text{target}(L_i) = 10^{i-1} \times 10 \text{ MiB}, \quad i \geq 1$$

so $L_1 \leq 10 \text{ MiB}$, $L_2 \leq 100 \text{ MiB}$, and so on. The LSM-Tree accepts *read amplification* as its trade-off: a key may live in multiple files across several levels. LevelDB mitigates this via Bloom filters and the non-overlapping key-range invariant for levels ≥ 1 .

The Write Path

When a caller invokes `db->Put(WriteOptions, key, value)`, the call is routed through `DBImpl::Write` in `db/db_impl.cc`. The public `DB::Put` method (defined in `db/db.cc`) is a thin wrapper that constructs a `WriteBatch` with a single `kTypeValue` record and calls `DBImpl::Write`. Similarly, `DB::Delete` constructs a single `kTypeDeletion` record. The full journey spans five distinct stages.

Stage 1 – Write-Group Formation (`DBImpl::Write`, `db/db_impl.cc`)

LevelDB serialises concurrent writers without per-write locking through a *write-group* mechanism defined in `DBImpl::Write` (`db/db_impl.cc`, around line 1112 in the base source). Every incoming `Put` or `Delete` is wrapped in a `WriteBatch` — a lightweight, append-only buffer of `(type, key, value)` records defined in `include/leveldb/write_batch.h`.

Each thread packages its `WriteBatch` into a `Writer` struct (defined in `db/db_impl.cc`) which holds a pointer to the batch, a condition variable `cv` backed by the global `mutex_`, and a `done` flag. The writer pushes itself onto the `writers_` deque (declared in `db/db_impl.h`) and waits on its condition variable until it reaches the head of the queue:

Listing 1: `db/db_impl.cc` – write-group serialisation

```
1 // Threads enqueue and wait until they are the queue leader
2 while (!w.done && w != writers_.front())
3     w.cv.Wait();
4 if (w.done) return w.status;    // another leader already handled this
    write
```

The thread at the front becomes the *group leader*. It calls `BuildBatchGroup(&last_writer)` (declared in `db/db_impl.h`, implemented in `db/db_impl.cc`) to absorb all pending writers'

batches into a single combined `WriteBatch`, subject to a size cap (`max_size` is bounded by 1 MiB plus the size of the first batch, or 128 KiB if the first batch is smaller):

Listing 2: `db/db_impl.cc` – `BuildBatchGroup` absorbs waiting writers

```
1 WriteBatch* updates = BuildBatchGroup(&last_writer);
2 WriteBatchInternal::SetSequence(updates, last_sequence + 1);
3 last_sequence += WriteBatchInternal::Count(updates);
```

`WriteBatchInternal::SetSequence` encodes the starting sequence number into the first 8 bytes of the batch's `rep_` string (defined in `db/write_batch_internal.h`). `WriteBatchInternal::Count` reads the 4-byte count field at bytes 8–11 of `rep_`, telling the group leader how many sequence numbers it will consume.

This amortises WAL I/O across arbitrarily many logical writes and eliminates per-write mutex contention. Once the leader finishes, it wakes every absorbed writer by marking `w.done = true` and calling `w.cv.Signal()`.

ForceFullCompaction write throttle (our addition). Our implementation extends `DBImpl::Write` to block foreground writes while a full compaction is running:

Listing 3: `db/db_impl.cc` – `Write()` throttle for `ForceFullCompaction`

```
1 MutexLock l(&mutex_);
2 // Block new foreground writes while ForceFullCompaction() is running
3 while (updates != nullptr && ShouldWaitFullCompaction()) {
4     background_work_finished_signal_.Wait();
5 }
6 Status usable_status = CheckDatabaseUsable();
7 if (!usable_status.ok()) return usable_status;
```

Only real writes (`updates != nullptr`) are blocked; the null-batch `Write` used internally by `FlushedMemTable` is deliberately exempt so the memtable flush can proceed without deadlock. Additionally, at the very end of `Write`, after notifying the next head of the writer queue, we added a `SignalAll` broadcast:

Listing 4: `db/db_impl.cc` – `Write()` signal broadcast at end

```
1 if (!writers_.empty())
2     writers_.front()->cv.Signal();
3 background_work_finished_signal_.SignalAll(); // NEW: wake
4     ForceFullCompaction waiters
5 return status;
```

This allows `ForceFullCompaction` (waiting on `ShouldWaitFullCompaction`) to be woken as soon as the preceding writes drain, reducing latency before the full compaction begins.

Stage 2 – Write-Ahead Log (`db/log_writer.h`, `db/log_writer.cc`)

Before touching memory the leader appends the combined `WriteBatch` to the **Write-Ahead Log** (WAL), a `.log` file managed by `log::Writer` (`db/log_writer.h`, `db/log_writer.cc`).

The WAL format is defined in `db/log_format.h`. Physical records are divided into fixed-size 32 KiB blocks. Each record header carries:

- A CRC32 checksum (bytes 0–3) for corruption detection.
- A 2-byte length field.
- A 1-byte `RecordType`: one of `kFullType`, `kFirstType`, `kMiddleType`, `kLastType` – allowing a logical record to span multiple physical blocks.

The write call in `DBImpl::Write` is:

Listing 5: `db/db_impl.cc` – WAL append

```

1 {
2     mutex_.Unlock();
3     status = log_>AddRecord(WriteBatchInternal::Contents(write_batch))
4         ;
5     bool sync_error = false;
6     if (status.ok() && options.sync) {
7         status = logfile_>Sync();
8         if (!status.ok()) sync_error = true;
9     }
10    if (status.ok()) {
11        status = WriteBatchInternal::InsertInto(write_batch, mem_);
12    }
13    mutex_.Lock();
14    ...
15 }
```

The mutex is *dropped* before the `log_>AddRecord` call so that concurrent reads and background compaction are not blocked during the I/O. `WriteOptions::sync` controls durability: with `sync=false` the write is acknowledged once data reaches the OS page cache; with `sync=true` the call blocks until `logfile_>Sync()` returns (i.e., `fdatasync` confirms stable-storage persistence).

`log::Writer::AddRecord` (`db/log_writer.cc`) fragments the batch data across 32 KiB block boundaries, computing the CRC32 checksum over the header and payload for each physical block using `crc32c::Value` and `crc32c::Extend` from `util/crc32c.h`. This fragmentation is transparent to the caller.

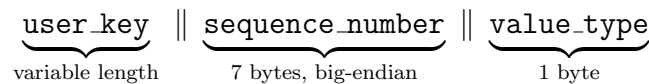
On crash recovery, `DBImpl::RecoverLogFile` in `db/db_impl.cc` reads each WAL record via `log::Reader` (`db/log_reader.cc`), decodes it via `WriteBatch::Iterate`, and replays it into a fresh `MemTable`, restoring all writes not yet flushed to SSTables. The `log::Reader` validates the CRC32 of each record and skips corrupted records when `paranoid_checks` is false, or returns an error if it is true.

Stage 3 – MemTable Insertion (`db/memtable.cc`, `db/skiplist.h`)

After WAL persistence, `WriteBatchInternal::InsertInto` (called inside the unlocked section above) creates a `MemTableInserter` handler (anonymous struct inside `db/write_batch.cc`) and calls `WriteBatch::Iterate`, which decodes each record from the batch's `rep_` string and dispatches to the handler. The handler's `Put` method calls `MemTable::Add` with `kTypeValue`, and its `Delete` method calls `MemTable::Add` with `kTypeDeletion` (`db/memtable.cc`).

The `MemTable` stores entries in a **lock-free skip list** (`db/skiplist.h`). Multiple reader threads may traverse the skip list simultaneously without any mutex; writes are serialised externally by the write-group mechanism.

Each entry is stored in **InternalKey** format (defined in `db/dbformat.h`):



The `MemTable::Add` function encodes this layout using `PutVarint32` from `util/coding.h`:

Listing 6: `db/memtable.cc` – `MemTable::Add` encoding

```

1 // key_size = internal_key_size
2 // key bytes = user_key + tag (sequence<<8 | type)
3 // val_size = value.size()
4 // val bytes = value.data()
5 size_t encoded_len = VarintLength(internal_key_size) +
    internal_key_size +
    VarintLength(val_size) + val_size;
6
7 char* buf = arena_.Allocate(encoded_len);
8 // ... encode varint(key_size), key bytes (user_key + tag),
9 //     varint(val_size), value ...
10 table_.Insert(buf);

```

The `arena_` is a `Arena` object (`util/arena.h`) that allocates memory in 4 KiB blocks and never frees individual entries, which is safe because the entire `MemTable` is discarded atomically when it becomes immutable.

The 56-bit monotonically-increasing `sequence_number` is the foundation of **MVCC**: readers pin a snapshot sequence number and see only entries whose sequence number is $\leq$ their snapshot. A deletion (`kTypeDeletion`) inserts a tombstone that shadows earlier versions; physical removal is deferred to compaction.

The internal comparator (`InternalKeyComparator` in `db/dbformat.cc`) orders entries by user key *ascending*, then by sequence number *descending* (newest first), so a seek for (`user_key`, `snapshot_seq`, `kValueTypeForSeek`) lands directly at the most recent visible version.

Our change: `kValueTypeForSeek` updated. We updated `kValueTypeForSeek` from `kTypeValue=0x1` to `kTypeRangeDeletion=0x2` in `db/dbformat.h`, since our new type is the numerically largest and must be used as the seek sentinel to ensure a seek lands at the most recent version of any key regardless of its type.

Stage 4 – *MemTable Rotation* (`MakeRoomForWrite`, `db/db_impl.cc`)

The `MemTable` grows until `mem_>ApproximateMemoryUsage()` exceeds `options_.write_buffer_size` (default 4 MiB). At that point `DBImpl::MakeRoomForWrite` (`db/db_impl.cc`) is called inside the write loop before the mutex is released for WAL I/O:

Listing 7: `db/db_impl.cc` – `MakeRoomForWrite`: memtable rotation

```

1 // Rotate: current memtable becomes immutable; allocate new memtable
  // + WAL
2 imm_ = mem_;
3 has_imm_.store(true, std::memory_order_release);
4 mem_ = new MemTable(internal_comparator_);
5 mem_>Ref();
6 force_log_compaction = true;
7 MaybeScheduleCompaction();

```


The full MemTable becomes the *Immutable MemTable* — a read-only snapshot that accepts no further writes. A new, empty MemTable backed by a fresh WAL file is allocated (the new WAL is opened via `env_>NewWritableFile` and wrapped in a `log::Writer`). The background compaction thread is then scheduled to flush the Immutable MemTable to a Level-0 SSTable.

`MakeRoomForWrite` also enforces back-pressure by consulting three thresholds (all defined as `constexpr` in `db/db_impl.cc`):

- It delays writes with a 1 ms sleep (`env_>SleepForMicroseconds(1000)`) when the Level-0 file count exceeds `kL0_SlowdownWritesTrigger = 8`.
- It halts writes entirely (`background_work_finished_signal_.Wait()`) when the count exceeds `kL0_StopWritesTrigger = 12`.
- It also stalls if an Immutable MemTable already exists (`imm_ != nullptr`), waiting for the background thread to flush it before allocating another.

Stage 5 – Flushing to an SSTable (*CompactMemTable*, `db/db_impl.cc`)

The background thread eventually calls `DBImpl::CompactMemTable`, which delegates to `BuildTable` (`db/builder.cc`). `BuildTable` iterates the Immutable MemTable in sorted InternalKey order via `mem->NewIterator()` and feeds each entry into a `TableBuilder` (`table/table_builder.cc`).

The resulting **SSTable** (`.ldb` file) has the following layout (defined in `table/format.h`):

Section	Contents	Purpose
Data blocks	Sorted (<code>InternalKey</code> , <code>value</code>) pairs	Actual storage
Index block	Last key + file offset per data block	Binary search
Filter block	Bloom filter per data block	Skip reads on miss
Metaindex block	Named handles (filter, range-del, ...)	Navigation
Footer (48 B)	Handles to index + metaindex blocks	File anchor

Data blocks use *prefix compression*: consecutive keys sharing a common prefix encode only the delta. Every 16th key (*restart point*) is stored verbatim, enabling $O(\log n)$ binary search within a block.

The footer is encoded by `Footer::EncodeTo` (`table/format.cc`) and consists of the encoded `BlockHandle` for the metaindex block, the encoded `BlockHandle` for the index block, and an 8-byte magic number `kTableMagicNumber` (0xdb4775248b80fb57) for file-format identification.

Once the SSTable is durably written, a `VersionEdit` is appended atomically to the **MANIFEST** log (`db/version_set.cc`), officially promoting the new file into the LSM hierarchy. The MANIFEST is itself a `log::Writer` instance, giving the same crash-safety properties as the WAL.

Our change: boundary expansion in `WriteLevel0Table`. After `BuildTable` returns, our implementation expands the file’s `FileMetaData` bounds to include any range tombstones stored in the Immutable MemTable’s `range_deletions_` list (see Section 3), because tombstone start/end keys may lie outside the point-key range.

The Read Path

A call to `db->Get(ReadOptions, key, &value)` resolves through `DBImpl::Get (db/db_impl.cc)`.

Snapshot Pinning and Version Reference

`DBImpl::Get` first acquires the mutex briefly to capture a snapshot sequence number and to obtain a reference to the current `Version`. Our implementation additionally waits here if a `ForceFullCompaction` is in progress:

Listing 8: `db/db_impl.cc` – `DBImpl::Get` prologue with FFC guard

```

1  MutexLock l(&mutex_);
2  while (ShouldWaitFullCompaction()) {
3      background_work_finished_signal_.Wait();
4  }
5  // ...
6  SequenceNumber snapshot;
7  if (options.snapshot != nullptr) {
8      snapshot = static_cast<const SnapshotImpl*>(options.snapshot)
9                ->sequence_number();
10 } else {
11     snapshot = versions_->LastSequence();
12 }
13
14 MemTable* mem = mem_;
15 MemTable* imm = imm_;
16 Version* current = versions_->current();
17 mem->Ref();
18 if (imm != nullptr) imm->Ref();
19 current->Ref();

```

The mutex is then released for the actual data lookups so that reads do not block concurrent writes.

Probe Order: MemTable, ImmMemTable, SSTables

The lookup proceeds in strictly newest-to-oldest order:

Listing 9: `db/db_impl.cc` – `DBImpl::Get` multi-layer probe

```

1  LookupKey lkey(key, snapshot);
2  Status s;
3  {
4      // 1. Active MemTable
5      if (mem->Get(lkey, value, &s)) { /* found or deleted */ }
6      // 2. Immutable MemTable (if present)
7      else if (imm != nullptr && imm->Get(lkey, value, &s)) { /* found */ }
8      // 3. SSTable levels (via current Version)
9      else {
10         s = current->Get(options, lkey, value, &stats);
11         have_stat_update = true;
12     }
13 }

```

MemTable::Get (db/memtable.cc). The **MemTable::Get** function encodes the query as a **LookupKey** (`user_key + sequence_number + kValueTypeForSeek`) and uses the skip list's **Seek** to land at the first entry $\leq$ the lookup key. Our extended implementation reads `read_seq` from the lookup key's 8-byte tag and processes entries in a **while** loop rather than a single **if**, to handle **kTypeRangeDeletion** skip-list markers:

Listing 10: db/memtable.cc – MemTable::Get (extended)

```

1  const Slice ikey = key.internal_key();
2  SequenceNumber read_seq = DecodeFixed64(ikey.data() + ikey.size()-8)
   >> 8;
3
4  while (iter.Valid()) {
5      // decode key_ptr, key_length ...
6      if (comparator_.comparator.Compare(...) == 0) {
7          ValueType type = static_cast<ValueType>(tag & 0xff);
8          SequenceNumber found_seq = tag >> 8;
9          const Comparator* ucmp = comparator_.comparator.user_comparator()
   ;
10         // Check 1: shadowed by a range tombstone?
11         if (range_deletions_.IsDeleted(key.user_key(), found_seq,
12             read_seq, ucmp)) {
13             *s = Status::NotFound(Slice());
14             return true;
15         }
16         switch (type) {
17             case kTypeValue: { /* return value */ }
18             case kTypeDeletion:
19                 *s = Status::NotFound(Slice()); return true;
20             case kTypeRangeDeletion:
21                 iter.Next(); continue; // skip marker; keep searching
22         }
23     } else { break; }
24 }
25 // Check 2: catch-all for keys with no explicit value entry
26 if (range_deletions_.IsDeleted(key.user_key(), 0, read_seq, ucmp)) {
27     *s = Status::NotFound(Slice()); return true;
28 }
29 return false;

```

Version::Get (db/version_set.cc). If neither MemTable hits, **Version::Get** searches the SSTable levels. For **Level 0**, every file whose key range overlaps the query key is sorted by sequence number (newest first) and probed individually — Level-0 files may have overlapping key ranges, so all must be checked. For **Levels 1–6**, the non-overlapping invariant means at most one file per level can contain the key; binary search on the file metadata (`version_set.cc: FindFile`) identifies it in $O(\log F)$.

Each candidate SSTable is probed via **TableCache::Get** (db/table_cache.cc), which calls **Table::InternalGet** (table/table.cc):

Listing 11: table/table.cc – Table::InternalGet SSTable probe (extended)

```

1  Slice user_key = ExtractUserKey(k);
2  SequenceNumber read_seq = DecodeFixed64(k.data() + k.size()-8) >> 8;
3  bool point_key_found = false;
4  // ... index block seek ...

```

```

5  Iterator* block_iter = BlockReader(this, options, iiter->value());
6  block_iter->Seek(k);
7  if (block_iter->Valid()) {
8      Slice found_user_key = ExtractUserKey(block_iter->key());
9      if (ucmp->Compare(found_user_key, user_key) == 0) {
10         point_key_found = true;
11         SequenceNumber found_seq = DecodeFixed64(
12             block_iter->key().data() + block_iter->key().size()-8) >> 8;
13         bool deleted = (rep->range_deletions != nullptr) &&
14             rep->range_deletions->IsDeleted(found_user_key,
15                                             found_seq, read_seq, ucmp);
16         if (!deleted)
17             (*handle_result)(arg, block_iter->key(), block_iter->value());
18         else
19             s = Status::NotFound(Slice());
20     }
21 }
22 // Catch-all: key shadowed by range tombstone with no point entry
23 if (!point_key_found && s.ok() && rep->range_deletions != nullptr &&
24     rep->range_deletions->IsDeleted(user_key, 0, read_seq, ucmp))
25     s = Status::NotFound(Slice());

```

The Bloom filter (default 10 bits/key, $\approx 1\%$ false-positive rate) avoids unnecessary data-block reads for keys absent from the file. Only on a filter hit is the data block read and the key sought within it.

Saver callback (db/version_set.cc). The `handle_result` callback is the `SaveValue` function inside `Version::Get`. It inspects the `ValueType` of the found `InternalKey` using a switch statement. Our extension adds `kDeletedByRange` to the `SaverState` enum and treats it identically to `kDeleted`:

Listing 12: db/version_set.cc – SaverState and SaveValue dispatch

```

1  enum SaverState { kNotFound, kFound, kDeleted, kCorrupt,
2                  kDeletedByRange };
3  // In SaveValue:
4  switch (parsed_key.type) {
5      case kTypeValue:      s->state = kFound;          break;
6      case kTypeDeletion:   s->state = kDeleted;         break;
7      case kTypeRangeDeletion: s->state = kDeletedByRange; break;
8      default:              s->state = kCorrupt;         break;
9  }
10 // In Version::Get's loop:
11 case kDeleted:
12 case kDeletedByRange:
13     return false; // stop searching

```

Iterator-Based Range Reads (DBIter, db/db_iter.cc)

`DBImpl::NewIterator` (db/db_impl.cc) is extended to wait for any in-progress `ForceFullCompaction` before creating an iterator:

Listing 13: db/db_impl.cc – DBImpl::NewIterator with FFC guard

```

1  Iterator* DBImpl::NewIterator(const ReadOptions& options) {
2      {

```

```

3     MutexLock l(&mutex_);
4     while (ShouldWaitFullCompaction())
5         background_work_finished_signal_.Wait();
6     if (shutting_down_.load(std::memory_order_acquire))
7         return NewErrorIterator(Status::IOError("Database is shutting
            down"));
8     }
9     SequenceNumber latest_snapshot;
10    uint32_t seed;
11    Iterator* iter = NewInternalIterator(options, &latest_snapshot, &
        seed);
12    return NewDBIterator(
13        this, user_comparator(), iter,
14        (options.snapshot != nullptr
15         ? static_cast<const SnapshotImpl*>(options.snapshot)
16           ->sequence_number()
17         : latest_snapshot),
18        seed);
19 }

```

`NewInternalIterator` (`db/db_impl.cc`) builds a `MergingIterator` (`table/merger.cc`) over:

1. The active MemTable's iterator.
2. The Immutable MemTable's iterator (if non-null).
3. One `TwoLevelIterator` per Level-0 file (since ranges may overlap).
4. One `TwoLevelIterator` per level ≥ 1 (using the level's concatenated file iterator from `Version::NewConcatenatingIterator`).

The `DBIter` wraps this internal `MergingIterator` and surfaces only live user-visible keys. `FindNextUserEntry` (`db/db_iter.cc`) advances the internal iterator and suppresses entries using three filters (detailed in Section 2).

Read Amplification

In the worst case a `Get` performs:

1. $O(\log M)$ skip-list probe in the active MemTable.
2. $O(\log M)$ probe in the Immutable MemTable.
3. Up to `kL0_CompactionTrigger` = 4 Bloom-filter + data-block probes in Level 0.
4. One Bloom-filter + data-block probe per Level 1–6.

Worst-case: $4 + 6 = 10$ Bloom-filter probes. A Bloom filter hit avoids the data-block I/O entirely; only a false positive triggers an unnecessary block read.

The Compaction Mechanism

Compaction is the LSM-Tree's background maintenance process: it merges SSTables across levels, physically reclaims deleted entries, and enforces the non-overlapping invariant for levels ≥ 1 . The entry point is `DBImpl::BackgroundCompaction` (`db/db_impl.cc`), called from the background thread via `DBImpl::BackgroundCall` $\rightarrow$ `DBImpl::BGWork` (a static trampoline).

Triggering: Score-Based Selection (*db/version_set.cc*)

`VersionSet::Finalize` (*db/version_set.cc*) computes a *score* for each level after every compaction:

$$\text{score}(L_0) = \frac{\text{file count}}{\text{kL0_CompactionTrigger}} \quad (1)$$

$$\text{score}(L_i) = \frac{\text{total bytes in } L_i}{\text{target}(L_i)}, \quad i \geq 1 \quad (2)$$

The level with the highest score > 1.0 is selected for compaction:

Listing 14: *db/version_set.cc* – Finalize: score computation

```

1 for (int level = 0; level < config::kNumLevels - 1; level++) {
2     double score;
3     if (level == 0) {
4         score = v->files_[0].size() /
5             static_cast<double>(config::kL0_CompactionTrigger);
6     } else {
7         const uint64_t level_bytes = TotalFileSize(v->files_[level]);
8         score = static_cast<double>(level_bytes)
9             / MaxBytesForLevel(&options_, level);
10    }
11    if (score > best_score) {
12        best_level = level;
13        best_score = score;
14    }
15 }
16 v->compaction_level_ = best_level;
17 v->compaction_score_ = best_score;

```

Within the selected level, a per-level `compact_pointer_` (*db/version_set.cc*) implements *round-robin* file selection, ensuring compaction sweeps evenly across the key space:

Listing 15: *db/version_set.cc* – PickCompaction round-robin

```

1 // Start with the file after compact_pointer_[level]
2 for (size_t i = 0; i < current_->files_[level].size(); i++) {
3     FileMetaData* f = current_->files_[level][i];
4     if (compact_pointer_[level].empty() ||
5         icmp_.Compare(f->largest.Encode(), compact_pointer_[level]) >
6         0) {
7         c->inputs_[0].push_back(f);
8         break;
9     }
10 }

```

MaybeScheduleCompaction bypass for ForceFullCompaction (our addition).

Our implementation suppresses routine auto-compaction scheduling while `ForceFullCompaction` is running, to prevent background work from interfering with the sequential level-draining:

Listing 16: *db/db_impl.cc* – MaybeScheduleCompaction FFC guard

```

1 } else if (force_full_compaction_in_progress_ &&
2            imm_ == nullptr &&
3            manual_compaction_ == nullptr) {

```

```

4 // suppress routine auto-compaction scheduling
5 }

```

Immutable-memtable flushes (`imm_ != nullptr`) and user-initiated manual compactions are still permitted through this guard to prevent deadlock.

Input Selection and Overlap Expansion (db/version_set.cc)

After selecting the seed file from level i , `VersionSet::SetupOtherInputs` (db/version_set.cc) finds all files from level $i+1$ whose key ranges overlap the seed file's range. To avoid creating unnecessarily many output files, it also expands the level- i input set (without increasing the level- $i+1$ overlap) if doing so costs less than `kExpandedCompactionByteSizeLimit`.

The Merge Process (DoCompactionWork, db/db_impl.cc)

The core merge is performed by `DBImpl::DoCompactionWork` (db/db_impl.cc). A `MergingIterator` performs an N -way heap merge over all input SSTables in sorted `InternalKey` order.

Listing 17: db/db_impl.cc – DoCompactionWork: merge loop skeleton

```

1 input->SeekToFirst();
2 while (input->Valid() && !shutting_down_.load(std::
   memory_order_acquire)) {
3   // 1. Check if immutable memtable needs flushing (priority flush)
4   if (has_imm_.load(std::memory_order_relaxed)) {
5     CompactMemTable();
6   }
7
8   Slice key = input->key();
9   // 2. Decide whether to drop or keep this entry
10  bool drop = false;
11  InternalKey ikey;
12  if (!ParseInternalKey(key, &ikey)) {
13    drop = false; // corrupt key: keep conservatively
14  } else {
15    if (!has_current_user_key ||
16        user_comparator()->Compare(ikey.user_key, current_user_key)
17        != 0) {
18      // First occurrence of this user_key: always keep
19      current_user_key.assign(ikey.user_key.data(), ikey.user_key.
20                             size());
21      has_current_user_key = true;
22      last_sequence_for_key = kMaxSequenceNumber;
23    }
24    if (last_sequence_for_key <= compact->smallest_snapshot) {
25      drop = true; // superseded by a newer version below the
26                  snapshot
27    } else if (ikey.type == kTypeDeletion &&
28               ikey.sequence <= compact->smallest_snapshot &&
29               compact->compaction->IsBaseLevelForKey(ikey.user_key))
30    {
31      drop = true; // obsolete tombstone: no older copies remain
32    }
33    last_sequence_for_key = ikey.sequence;
34  }
35  if (!drop) {

```



```

32     if (compact->builder == nullptr) OpenCompactionOutputFile(compact
33         );
34     compact->builder->Add(key, input->value());
35     ...
36 }
37 input->Next();
38 }

```

Key-dropping rules applied during the merge:

1. **Superseded versions:** for a given user key, only the entry with the highest sequence number survives; all older versions are dropped.
2. **Obsolete point tombstones:** a `kTypeDeletion` entry whose sequence number is $\leq$ the oldest live snapshot and for which no older copy exists in higher levels (`IsBaseLevelForKey`) is dropped.
3. **Range-deleted entries** (our extension): entries covered by a range tombstone whose sequence number is $\leq$ the oldest snapshot are dropped (see Section 3).

Output file sizing. Output SSTables are capped at `options.max_file_size` (default 2 MiB) or when estimated overlap with level $i+2$ exceeds `kMaxGrandparentOverlapBytes` (20 MiB), preventing future compactions from being excessively large.

Atomic commit. Completion is committed by calling `InstallCompactionResults` (`db/db_impl.cc`), which appends a single `VersionEdit` to the MANIFEST and calls `versions->LogAndApply`. A crash before this append leaves the database in the prior consistent state.

Trivial Move (db/db_impl.cc – BackgroundCompaction)

When the selected level- i file has no overlap with level- $i+1$, `BackgroundCompaction` performs a *trivial move*: the file's metadata is transferred to level $i+1$ in a single `VersionEdit` without any I/O:

Listing 18: `db/db_impl.cc` – trivial move

```

1  if (c->IsTrivialMove()) {
2      assert(c->num_input_files(0) == 1);
3      FileMetaData* f = c->input(0, 0);
4      c->edit()->RemoveFile(c->level(), f->number);
5      c->edit()->AddFile(c->level() + 1, f->number, f->file_size,
6                      f->smallest, f->largest);
7      versions->LogAndApply(c->edit(), &mutex_);
8      RecordForceCompactionStats(1, 1, f->file_size, f->file_size);
9  }

```

Our addition of `RecordForceCompactionStats` records this trivial move as a single-file compaction event when `ForceFullCompaction` is active.

Memtable-Flush Priority

During `DoCompactionWork`, the loop checks `has_imm.load(std::memory_order_relaxed)` at the start of each iteration. If the Immutable MemTable is non-null, the compaction is interrupted, `CompactMemTable()` is called to flush it to Level 0, and then the inter-level

compaction resumes. This ensures the immutable buffer does not accumulate indefinitely while a slow compaction is running.

Write Amplification

A byte written by the application is amplified as follows:

1. Written once to the WAL (`log::Writer`).
2. Flushed once as a Level-0 SSTable (`CompactMemTable`).
3. Compacted from Level-0 to Level-1 ($\approx 10\times$ amplification due to the size ratio).
4. Compacted per additional level ($\approx 10\times$ per level).

Total write amplification to reach level k is approximately $2 + 10(k - 1)$, giving $\approx 50\times$ for a 6-level hierarchy in the worst case. In practice most data stabilises after 2–3 levels.

Key Design Observations

Write-group leader pattern. `BuildBatchGroup` in `db/db_impl.cc` amortises WAL I/O across an arbitrary number of concurrent writers with zero per-write mutex contention. The pattern is a classic producer-consumer optimisation: the group leader acts as a batch processor for all concurrently waiting writers.

Lock-free skip list. The `SkipList` in `db/skiplist.h` uses `std::atomic` load/store with `memory_order_acquire` / `memory_order_release` to allow multiple concurrent readers without any mutex. Writes are safe because the write-group mechanism ensures only one writer inserts at a time. This is the primary reason LevelDB achieves high read concurrency even under heavy write load.

Immutability as synchronisation. Freezing the current MemTable into an Immutable MemTable (`imm_`) allows the background flush thread to drain it to disk concurrently with foreground writes into the new `mem_`. The `has_imm` atomic flag (`std::memory_order_release` on set, `std::memory_order_relaxed` on check) provides the necessary visibility guarantee without a mutex.

Append-only MANIFEST. Recording compaction outcomes as append-only `VersionEdit` log entries in `VersionSet::LogAndApply` (`db/version_set.cc`) achieves crash-safe metadata transitions. A partial write to the MANIFEST is detected on recovery because the record checksum (from `log::Reader` in `db/log_reader.cc`) will not match, and the last incomplete edit is discarded.

Snapshot isolation via sequence numbers. Because every `IteratorWrapper` inside the `MergingIterator` exposes `InternalKeys`, and `DBIter::ParseKey` filters by `sequence <= snapshot`, a reader sees a completely stable point-in-time view regardless of concurrent writes. This is identical in semantics to MVCC snapshot isolation in relational databases, implemented here with $O(1)$ overhead per key.

Deferred reorganisation and back-pressure. The score-based, round-robin compaction scheduling in `VersionSet::Finalize` and `VersionSet::PickCompaction` spreads maintenance work evenly. However, a write-heavy workload can accumulate Level-0 files

faster than compaction can drain them. The back-pressure thresholds — `kL0_SlowdownWritesTrigger=8` and `kL0_StopWritesTrigger=12` in `db/db_impl.cc` — prevent unbounded accumulation at the cost of write latency spikes. Tuning `write_buffer_size`, `max_open_files`, and compression type is therefore essential for production deployments.

Design and Implementation of the RangeScan API

Motivation

LevelDB's public interface provides only point lookups (`Get`) and manual iteration (`NewIterator`). Many applications require atomically retrieving *all* key-value pairs in a contiguous key interval. The `Scan` API encapsulates this common pattern inside the engine, guaranteeing snapshot consistency, correct MVCC filtering, and multi-level visibility without requiring callers to manage iterators directly.

Public API Changes

include/leveldb/db.h

Three standard library headers are added (`<string>`, `<utility>`, `<vector>`) to support the new return type. Two new pure-virtual declarations are added to the abstract `DB` class:

Listing 19: `include/leveldb/db.h` – new virtual declarations

```

1 virtual Status Scan(
2     const ReadOptions& options,
3     const Slice& start_key,
4     const Slice& end_key,
5     std::vector<std::pair<std::string, std::string>>* result) = 0;
6
7 virtual Status DeleteRange(const WriteOptions& options,
8                             const Slice& start_key,
9                             const Slice& end_key) = 0;
10
11 // (also added below CompactRange)
12 virtual Status ForceFullCompaction() = 0;
```

The signature of `Scan` follows the same convention as `Get`: it takes a `ReadOptions` struct (which may carry an optional snapshot pointer and a fill-cache flag), the half-open range endpoints as `Slice` objects, and an output-parameter vector. Returning a `Status` lets the caller distinguish I/O errors from an empty range without exceptions.

db/db_impl.h

The concrete class `DBImpl` declares the overrides:

Listing 20: `db/db_impl.h` – `DBImpl` overrides

```

1 Status Scan(const ReadOptions& options,
2             const Slice& start_key, const Slice& end_key,
3             std::vector<std::pair<std::string, std::string>>* result)
4             override;
5
6 Status DeleteRange(const WriteOptions& options,
7                   const Slice& start_key,
```

```

7         const Slice& end_key) override;
8
9 void GetRangeDeletions(RangeDeletionList* list);
10
11 Status ForceFullCompaction() override;

```

`GetRangeDeletions` is a helper (not part of the public `DB` interface) used by `DBIter` to pre-fetch the complete set of range tombstones on iterator construction.

Implementation – `DBImpl::Scan` (`db/db_impl.cc`)

Full-Compaction Guard

Before entering the scan proper, the implementation acquires the mutex briefly to check whether a `ForceFullCompaction` is in progress (via `ShouldWaitFullCompaction()`) and whether the database is healthy (via checking for shutdown):

Listing 21: `db/db_impl.cc` – `DBImpl::Scan` FFC guard

```

1 {
2     MutexLock l(&mutex_);
3     while (ShouldWaitFullCompaction())
4         background_work_finished_signal_.Wait();
5     if (shutting_down_.load(std::memory_order_acquire))
6         return Status::IOError("Database is shutting down");
7 }

```

This ensures the scan sees a stable, fully-compacted view and is not interleaved with level-draining operations.

Core Scan Loop

Listing 22: `db/db_impl.cc` – `DBImpl::Scan` implementation

```

1 Status DBImpl::Scan(const ReadOptions& options,
2                    const Slice& start_key, const Slice& end_key,
3                    std::vector<std::pair<std::string, std::string>>*
4                    result) {
5     { /* FFC guard above */ }
6     result->clear();
7     if (start_key.compare(end_key) >= 0) // empty range: nothing to
8         do
9         return Status::OK();
10
11     Iterator* it = this->NewIterator(options);
12     it->Seek(start_key);
13     while (it->Valid() && it->key().compare(end_key) < 0) {
14         result->emplace_back(it->key().ToString(), it->value().ToString());
15         it->Next();
16     }
17     Status status = it->status();
18     delete it;
19     return status;
20 }

```

Key properties:

- **Empty-range fast path:** `start_key >= end_key` returns immediately without allocating an iterator.
- **Snapshot consistency:** `NewIterator(options)` creates a `DBIter` that pins a snapshot sequence number at creation time.
- **MVCC and tombstone filtering:** All filtering (dead versions, point tombstones, range tombstones) is handled transparently inside `DBIter`.
- **Multi-level visibility:** The internal `MergingIterator` merges the active `MemTable`, the `Immutable MemTable` (if present), and every `SSTable` level via `TableCache::NewIterator`.
- **Mutex not held during I/O:** The lock is released before `NewIterator` is called, so the scan does not block concurrent writes.

Role of DBIter

`NewIterator` returns a `DBIter` instance created by `NewDBIterator (db/db_iter.cc)`. `DBIter` wraps the internal `MergingIterator` and presents only live user-visible keys to the caller.

Constructor Change – Pre-Fetching Range Tombstones

The `DBIter` constructor is extended to call `db->GetRangeDeletions(&global_range_dels_)`:

Listing 23: `db/db_iter.cc` – `DBIter` constructor extension

```

1 DBIter::DBIter(DBImpl* db, const Comparator* cmp, Iterator* iter,
2               SequenceNumber s, uint32_t seed)
3   : db_(db), user_comparator_(cmp), iter_(iter),
4     sequence_(s), direction_(kForward), valid_(false),
5     rnd_(seed),
6     bytes_until_read_sampling_(RandomCompactionPeriod()) {
7   // NEW: collect ALL range tombstones across the entire DB at
8   //       creation time
9   db->GetRangeDeletions(&global_range_dels_);
10 }

```

The new data member `global_range_dels_` of type `RangeDeletionList` is also added to the `DBIter` class. Pre-fetching all tombstones once at iterator-creation time means every key checked during a scan (or any other iteration) is validated against the complete set of range tombstones with *zero* per-key I/O overhead.

FindNextUserEntry – Three Suppression Filters

After each `Seek` or `Next`, `DBIter` calls `FindNextUserEntry` to skip any keys that must not be exposed. The function is extended to handle `kTypeRangeDeletion` marker entries and to apply the range-tombstone check for every `kTypeValue` candidate. Note also that valid-key handling now explicitly saves both the key and value before setting `valid_ = true`, which is a correctness fix compared to the original code:

Listing 24: `db/db_iter.cc` – `FindNextUserEntry` (extended)

```

1 case kTypeRangeDeletion:
2   // Marker is in the internal key stream; boundary logic is
3   // handled entirely by global_range_dels_ -- skip the marker.
4   break;
5
6 case kTypeValue:

```

```

7   if (global_range_dels_.IsDeleted(ikey.user_key, ikey.sequence,
8                                   sequence_, user_comparator_)) {
9       // Shadowed by global Range Tombstone
10  } else if (skipping &&
11             user_comparator_>Compare(ikey.user_key, *skip) <= 0) {
12       // Shadowed by Point Tombstone
13  } else {
14       // Valid key: save key+value, then mark valid
15       saved_key_.clear();
16       SaveKey(ikey.user_key, &saved_key_);
17       saved_value_.assign(iter_>value().data(), iter_>value().size())
18       ;
19       valid_ = true;
20       return;
21  }
22  break;

```

The three suppression filters are:

1. **Sequence filter:** entries with `ikey.sequence > sequence_` are already skipped by `ParseKey`.
2. **Range tombstone:** `global_range_dels_.IsDeleted` checks whether the key falls within any active range tombstone with the correct MVCC semantics. The `user_comparator_` is passed explicitly to ensure correct lexicographic ordering (Section 3).
3. **Point tombstone:** the `skipping` flag suppresses older versions of a key after a `kTypeDeletion` entry is seen.

FindPrevUserEntry – Reverse Iteration

For reverse iteration (`Prev()`), `FindPrevUserEntry` is also extended with the range-tombstone check:

Listing 25: `db/db_iter.cc` – `FindPrevUserEntry` range check

```

1 // Check if shadowed by a global range tombstone
2 if (global_range_dels_.IsDeleted(ikey.user_key, ikey.sequence,
3                                   sequence_, user_comparator_)) {
4     value_type = kTypeDeletion; // treat exactly like a point
5     tombstone
6 }
7 if (value_type == kTypeDeletion || value_type == kTypeRangeDeletion)
8 {
9     saved_key_.clear();
10    ClearSavedValue();
11 } else { ... }

```

This ensures that reverse scans and `Prev()`-based traversals are equally range-deletion aware.

DBImpl::GetRangeDeletions (`db/db_impl.cc`)

This new function aggregates range tombstones from all live data sources. It acquires the mutex for the entire call to prevent races with concurrent memtable rotations or version changes:

Listing 26: db/db_impl.cc – GetRangeDeletions

```

1 void DBImpl::GetRangeDeletions(RangeDeletionList* list) {
2     MutexLock l(&mutex_);
3     if (mem_)
4         list->MergeInto(mem_->GetRangeDeletions());
5     if (imm_)
6         list->MergeInto(imm_->GetRangeDeletions());
7     if (versions_ && versions_->current()) {
8         Version* current = versions_->current();
9         for (int level = 0; level < config::kNumLevels; level++) {
10             size_t i = 0;
11             const auto& files = current->files_[level];
12             while (i < files.size()) {
13                 FileMetaData* f = files[i];
14                 table_cache_->GetRangeDeletions(f->number, f->file_size, list);
15                 i++;
16             }
17         }
18     }
19 }

```

The function chains: (1) the active MemTable’s in-memory tombstone list, (2) the Immutable MemTable’s list (if non-null), (3) every SSTable in the current version via `TableCache::GetRangeDeletions` (which reuses the LRU table cache – no extra file I/O if the file is already open). Note that `DBImpl` is declared as a `friend` of `Version` (`db/version_set.h`) to allow direct access to the `files_` array.

Snapshot Isolation and Correctness

Because the sequence number is fixed at `DBIter` creation time and applied in every `ParseKey` call inside the internal iterator, the scan produces a *snapshot-consistent* result: keys inserted after the snapshot point are invisible even if they fall within `[start_key, end_key)`. This is the same guarantee provided by LevelDB’s point `Get`.

Complexity

Let N = keys in range, T = total range tombstones.

- **Time:** $O(N \cdot T)$ worst-case because `IsDeleted` scans the tombstone list linearly per key. For typical workloads where T is small this is effectively $O(N)$.
- **Space:** $O(N)$ for the result vector; $O(T)$ for the tombstone list (shared with the iterator).

Design and Implementation of the DeleteRange API

Motivation

Point deletions in LevelDB insert a single tombstone per key. Deleting a large contiguous key range via individual `Delete` calls is $O(N)$ in the number of affected keys and produces N tombstones that each consume space and must be individually processed during compaction. `DeleteRange` solves this by recording a *single* range tombstone that marks

the entire interval $[start_key, end_key)$ as deleted at a given sequence number, regardless of how many keys fall within it.

New Value Type and Thread-Safe Structures – db/dbformat.h

The foundational change is adding a new internal-key type and thread-safe data structures to db/dbformat.h. The header also gains `<mutex>` and `<vector>` includes.

Extended ValueType Enum

Listing 27: db/dbformat.h – extended ValueType enum

```
1 enum ValueType {
2     kTypeDeletion      = 0x0,
3     kTypeValue         = 0x1,
4     kTypeRangeDeletion = 0x2    // NEW
5 };
6
7 // kValueTypeForSeek must be the highest-numbered type so that a seek
8 // to (user_key, snapshot_seq, kValueTypeForSeek) lands at the most
9 // recent version of the key.
10 // Updated from kTypeValue to kTypeRangeDeletion:
11 static const ValueType kValueTypeForSeek = kTypeRangeDeletion;
```

Because internal keys are ordered by user key ascending then sequence number *descending* (with value type as tie-breaker), `kValueTypeForSeek` must be the numerically largest type. With `kTypeRangeDeletion=0x2` now the largest, it replaces `kTypeValue=0x1` as the seek type.

`ParseInternalKey` (also in db/dbformat.h) is updated to accept types up to `kTypeRangeDeletion`:

Listing 28: db/dbformat.h – ParseInternalKey acceptance range

```
1 return (c <= static_cast<uint8_t>(kTypeRangeDeletion));
2 // was: return (c <= static_cast<uint8_t>(kTypeValue));
```

RangeDeletion and RangeDeletionList – db/dbformat.h and db/dbformat.cc

Two new types are added to db/dbformat.h. A key design principle is that `RangeDeletionList` is **thread-safe by design**: it embeds a mutable `std::mutex mutex_` and acquires it in every public method. This is critical because `RangeDeletionList` instances are read by iterator threads and written by the write path concurrently.

Listing 29: db/dbformat.h – RangeDeletion and RangeDeletionList

```
1 struct RangeDeletion {
2     std::string start_key;
3     std::string end_key;
4     SequenceNumber seq;
5 };
6
7 class RangeDeletionList {
8 public:
9     RangeDeletionList() = default;
10    void Add(const Slice& start, const Slice& end, SequenceNumber seq);
11    bool IsDeleted(const Slice& key, SequenceNumber found_seq,
```

```

12         SequenceNumber read_seq,
13         const Comparator* ucmp = nullptr) const;
14 void MergeInto(const RangeDeletionList* other);
15 std::vector<RangeDeletion> GetDeletions() const;    // returns by-
16         value copy
17 private:
18     std::vector<RangeDeletion> deletions_;
19     mutable std::mutex mutex_;    // protects deletions_ for concurrent
20         access
};

```

RangeDeletionList::Add (db/dbformat.cc)

Listing 30: db/dbformat.cc – RangeDeletionList::Add

```

1 void RangeDeletionList::Add(const Slice& start, const Slice& end,
2                             SequenceNumber seq) {
3     std::lock_guard<std::mutex> lock(mutex_);
4     RangeDeletion del;
5     del.start_key = start.ToString();
6     del.end_key   = end.ToString();
7     del.seq       = seq;
8     deletions_.push_back(del);
9 }

```

The `std::lock_guard` ensures the push is atomic with respect to any concurrent reader calling `IsDeleted` or `MergeInto`.

RangeDeletionList::IsDeleted – MVCC Semantics and Comparator Injection (db/dbformat.cc)

Listing 31: db/dbformat.cc – IsDeleted with MVCC semantics

```

1 bool RangeDeletionList::IsDeleted(const Slice& key,
2                                   SequenceNumber found_seq,
3                                   SequenceNumber read_seq,
4                                   const Comparator* ucmp) const {
5     std::lock_guard<std::mutex> lock(mutex_);
6     for (const auto& del : deletions_) {
7         int start_cmp = ucmp ? ucmp->Compare(key, del.start_key)
8                             : key.compare(del.start_key);
9         int end_cmp   = ucmp ? ucmp->Compare(key, del.end_key)
10                                : key.compare(del.end_key);
11         if (start_cmp >= 0 && end_cmp < 0) {
12             if (del.seq <= read_seq && del.seq > found_seq)
13                 return true;
14         }
15     }
16     return false;
17 }

```

The optional `ucmp` (user comparator) parameter is a key design decision. When `ucmp` is provided, all range comparisons use the user-defined comparator rather than raw byte comparison. This is essential for correctness when a custom comparator is in use (e.g.,

a reverse-order comparator or a numeric comparator). The dual condition encodes full MVCC semantics:

- `del.seq <= read.seq`: the tombstone was written before the reader’s snapshot – it is visible.
- `del.seq > found.seq`: the tombstone is newer than the specific version of the key found – it overrides it.

This handles the write-after-delete-range case: if a key is written *after* the range tombstone ($S_v > S_r$), the condition `del.seq > found.seq` fails and the key is correctly visible.

RangeDeletionList::MergeInto (db/dbformat.cc)

Listing 32: db/dbformat.cc – MergeInto with dual-lock

```
1 void RangeDeletionList::MergeInto(const RangeDeletionList* other) {
2     if (other == nullptr || this == other) return;
3     // Lock in consistent order to avoid deadlock (other first, then
4     // this)
5     std::lock_guard<std::mutex> lock_other(other->mutex_);
6     std::lock_guard<std::mutex> lock_this(mutex_);
7     deletions_.insert(deletions_.end(),
8                       other->deletions_.begin(), other->deletions_.end());
9 }
```

A critical design principle here is the self-check (`this == other`) before acquiring locks, which prevents a trivial deadlock. The dual `lock_guard` pattern acquires `other->mutex_` first and then `mutex_` to establish a consistent lock ordering and avoid ABBA deadlocks.

RangeDeletionList::GetDeletions (db/dbformat.cc)

Listing 33: db/dbformat.cc – GetDeletions returns by-value copy

```
1 std::vector<RangeDeletion> RangeDeletionList::GetDeletions() const {
2     std::lock_guard<std::mutex> lock(mutex_);
3     return deletions_; // returns a copy, releasing the lock before
4                       // the caller iterates
5 }
```

Returning by value (rather than by const reference) means the caller can freely iterate over the copy without holding the mutex, which is important for the compaction loop in `DoCompactionWork` that iterates tombstones many times.

Public API Changes

`include/leveldb/db.h`

`DeleteRange` is declared as a pure virtual (shown above alongside `Scan`).

`include/leveldb/write_batch.h`

Two additions are made to `WriteBatch`:

1. A default no-op `DeleteRange` virtual in the `Handler` base class, so existing handlers (e.g., replication writers) compile unmodified.
2. A public `void DeleteRange(const Slice&, const Slice&)` method on `WriteBatch` itself.

Listing 34: include/leveldb/write_batch.h – changes

```

1 // In class Handler:
2 virtual void DeleteRange(const Slice& start_key, const Slice& end_key
  ) {}
3
4 // New public method on WriteBatch:
5 void DeleteRange(const Slice& start_key, const Slice& end_key);

```

Write Path – WriteBatch Encoding (db/write_batch.cc)

WriteBatch::DeleteRange

Listing 35: db/write_batch.cc – WriteBatch::DeleteRange serialisation

```

1 void WriteBatch::DeleteRange(const Slice& start_key, const Slice&
  end_key) {
2   WriteBatchInternal::SetCount(this, WriteBatchInternal::Count(this)
    + 1);
3   rep_.push_back(static_cast<char>(kTypeRangeDeletion));
4   PutLengthPrefixedSlice(&rep_, start_key);
5   PutLengthPrefixedSlice(&rep_, end_key);
6 }

```

The byte layout is: tag byte (0x2), length-prefixed start key, length-prefixed end key – analogous to `kTypeValue` entries.

WriteBatch::Iterate Dispatch

The `Iterate` loop gains a new case:

Listing 36: db/write_batch.cc – Iterate dispatch for range deletions

```

1 case kTypeRangeDeletion:
2   if (GetLengthPrefixedSlice(&input, &key) &&
3       GetLengthPrefixedSlice(&input, &value)) {
4     handler->DeleteRange(key, value);
5   } else {
6     return Status::Corruption("bad WriteBatch DeleteRange");
7   }
8   break;

```

MemTableInserter::DeleteRange

The anonymous `MemTableInserter` class (inside `db/write_batch.cc`) overrides `DeleteRange` to write the tombstone into the MemTable:

Listing 37: db/write_batch.cc – MemTableInserter::DeleteRange

```

1 void DeleteRange(const Slice& start_key, const Slice& end_key)
  override {
2   mem_->Add(sequence_, kTypeRangeDeletion, start_key, end_key);
3   sequence_++;
4 }

```

Each tombstone consumes exactly one sequence number, preserving the invariant that every skip-list entry has a unique sequence number.

MemTable Storage – db/memtable.h and db/memtable.cc

memtable.h Changes

A `RangeDeletionList` `range_deletions_` member is added to `MemTable`, and an accessor is provided:

Listing 38: db/memtable.h – new member and accessor

```
1 const RangeDeletionList* GetRangeDeletions() const {
2     return &range_deletions_;
3 }
4 // private:
5 RangeDeletionList range_deletions_;
```

MemTable::Add Hook (memtable.cc)

Listing 39: db/memtable.cc – MemTable::Add range-deletion hook

```
1 table_.Insert(buf); // always insert into skip list for WAL replay
   / SSTable flush
2
3 if (type == kTypeRangeDeletion) {
4     range_deletions_.Add(key, value, s); // also index in the fast
   tombstone list
5 }
```

Storing tombstones in the separate `RangeDeletionList` provides $O(1)$ amortised insertion and allows `Get` and iterator construction to consult tombstones without re-scanning the skip list. The `Add` call into `range_deletions_` is thread-safe by virtue of `RangeDeletionList`'s internal mutex.

DBImpl::DeleteRange (db/db_impl.cc)

Listing 40: db/db_impl.cc – DBImpl::DeleteRange

```
1 Status DBImpl::DeleteRange(const WriteOptions& options,
2                             const Slice& start_key,
3                             const Slice& end_key) {
4     if (user_comparator()->Compare(start_key, end_key) >= 0)
5         return Status::OK(); // empty range: no-op
6     WriteBatch batch;
7     batch.DeleteRange(start_key, end_key);
8     return Write(options, &batch);
9 }
```

An important design detail: the empty-range check uses `user_comparator()->Compare` rather than the raw `Slice::compare`, which ensures correctness when a custom comparator is in use (e.g., a comparator that ignores case or operates on a different collation order). Otherwise the tombstone is encoded into a `WriteBatch` and submitted through `Write`, traversing the exact same write-group, WAL, and MemTable path as any other write.

SSTable Persistent Path

include/leveldb/table.h Changes

`RangeDeletionList` is forward-declared; two new private/public members are added to `Table`:

Listing 41: `include/leveldb/table.h` – Table additions

```

1 // Forward declaration added at top of file
2 class RangeDeletionList;
3
4 // New public accessor:
5 RangeDeletionList* GetRangeDeletions() const;
6
7 // New private method:
8 void ReadRangeDeletions(const Slice& handle_value);

```

Table::Rep and Rep Destructor (table/table.cc)

`RangeDeletionList* range_deletions = nullptr` is added to `Table::Rep`, and its destructor is extended to `delete range_deletions`.

Table::ReadMeta Restructure (table/table.cc)

The original `ReadMeta` returned immediately if no filter policy was configured. This prevented range-deletion metadata from being loaded. It is restructured to *always* read the metaindex block, then separately look up the filter and range-deletion keys:

Listing 42: `table/table.cc` – `ReadMeta` restructured

```

1 void Table::ReadMeta(const Footer& footer) {
2   // No early-return for missing filter policy
3   ReadOptions opt;
4   if (rep_>options.paranoid_checks) opt.verify_checksums = true;
5   BlockContents contents;
6   if (!ReadBlock(rep_>file, opt, footer.metaindex_handle(), &
7     contents).ok())
8     return;
9
10  Block* meta = new Block(contents);
11  Iterator* iter = meta->NewIterator(BytewiseComparator());
12
13  // Filter block (conditional on filter policy)
14  if (rep_>options.filter_policy != nullptr) {
15    std::string filter_key = "filter.";
16    filter_key.append(rep_>options.filter_policy->Name());
17    iter->Seek(filter_key);
18    if (iter->Valid() && iter->key() == Slice(filter_key))
19      ReadFilter(iter->value());
20  }
21
22  // Range-deletion block (always)
23  std::string range_del_key = "leveldb.range_deletions";
24  iter->Seek(range_del_key);
25  if (iter->Valid() && iter->key() == Slice(range_del_key))
26    ReadRangeDeletions(iter->value());

```

```

26
27     delete iter;
28     delete meta;
29 }

```

Table::ReadRangeDeletions (table/table.cc)

Listing 43: table/table.cc – ReadRangeDeletions

```

1 void Table::ReadRangeDeletions(const Slice& handle_value) {
2     Slice v = handle_value;
3     BlockHandle handle;
4     if (!handle.DecodeFrom(&v).ok()) return;
5     ReadOptions opt;
6     if (rep_>options.paranoid_checks) opt.verify_checksums = true;
7     BlockContents block;
8     if (!ReadBlock(rep_>file, opt, handle, &block).ok()) return;
9
10    Block* range_del_block = new Block(block);
11    rep_>range_deletions = new RangeDeletionList();
12    Iterator* iter = range_del_block->NewIterator(BytewiseComparator())
13    ;
14    for (iter->SeekToFirst(); iter->Valid(); iter->Next()) {
15        Slice ikey      = iter->key();    // internal key: user_key + 8-
16        byte tag
17        Slice end_key = iter->value();    // end key stored as the "value"
18        if (ikey.size() < 8) continue;
19        const uint64_t tag = DecodeFixed64(ikey.data() + ikey.size() - 8)
20        ;
21        SequenceNumber seq = tag >> 8;
22        Slice start_key(ikey.data(), ikey.size() - 8); // strip the tag
23        rep_>range_deletions->Add(start_key, end_key, seq);
24    }
25    delete iter;
26    delete range_del_block;
27 }

```

dbformat.h is included in table/table.cc to access `SequenceNumber`, `DecodeFixed64`, and `RangeDeletionList`.

Table::GetRangeDeletions (table/table.cc)

Listing 44: table/table.cc – Table::GetRangeDeletions

```

1 RangeDeletionList* Table::GetRangeDeletions() const {
2     return rep_>range_deletions;
3 }

```

Table::InternalGet Changes (table/table.cc)

`InternalGet` is extended to enforce range tombstones during SSTable point lookups. The user comparator is extracted from the internal key comparator cast:

Listing 45: table/table.cc – InternalGet preamble and range check

```

1 Slice user_key      = ExtractUserKey(k);

```



```

2 SequenceNumber read_seq = DecodeFixed64(k.data() + k.size()-8) >> 8;
3 bool point_key_found = false;
4
5 // Inside block iterator loop:
6 const Comparator* ucmp = static_cast<const InternalKeyComparator*>(
7     rep->options.comparator->user_comparator());
8 if (ucmp->Compare(found_user_key, user_key) == 0) {
9     point_key_found = true;
10    bool deleted = (rep->range_deletions != nullptr) &&
11        rep->range_deletions->IsDeleted(found_user_key, found_seq,
12            read_seq, ucmp);
13    if (!deleted)
14        (*handle_result)(arg, block_iter->key(), block_iter->value());
15    else
16        s = Status::NotFound(Slice());
17 }
18 // Catch-all tombstone check after the main loop:
19 if (!point_key_found && s.ok() && rep->range_deletions != nullptr &&
20     rep->range_deletions->IsDeleted(user_key, 0, read_seq, ucmp))
21     s = Status::NotFound(Slice());

```

TableBuilder Changes (table/table_builder.cc)

New range_del_block Member

TableBuilder::Rep gains a BlockBuilder range_del_block member, initialised in the constructor alongside data_block and index_block.

TableBuilder::Add – Routing Range Tombstones

The Add method is extended to detect and route range-deletion entries:

Listing 46: table/table_builder.cc – TableBuilder::Add routing

```

1 size_t n = key.size();
2 if (n >= 8) {
3     const uint64_t tag = DecodeFixed64(key.data() + n - 8);
4     ValueType type = static_cast<ValueType>(tag & 0xff);
5     if (type == kTypeRangeDeletion) {
6         r->range_del_block.Add(key, value);
7         r->num_entries++;
8         return; // do NOT add to data_block, filter, or index
9     }
10 }
11 // ... normal data-block path ...

```

By routing tombstones into the dedicated range_del_block, they do not pollute data blocks, do not generate index entries, and do not affect Bloom filter construction.

The guard on the sort-order assertion is also relaxed:

Listing 47: table/table_builder.cc – assertion guard

```

1 if (r->num_entries > 0 && !r->last_key.empty()) {
2     assert(r->options.comparator->Compare(key, Slice(r->last_key)) > 0)
3     ;
4 }

```

```
4 // was: if (r->num_entries > 0) { assert(...) }
```

This avoids a false assertion when the first real data-block entry follows a range-deletion entry (which does not set `last_key`).

TableBuilder::Finish – Persisting the Range Deletion Block

Listing 48: table/table_builder.cc – Finish: writing range-deletion block

```
1 BlockHandle range_del_block_handle;
2 bool has_range_dels = !r->range_del_block.empty();
3
4 if (ok() && has_range_dels)
5     WriteBlock(&r->range_del_block, &range_del_block_handle);
6
7 // ... filter block, then metaindex block ...
8 if (has_range_dels) {
9     std::string key = "leveldb.range_deletions";
10    std::string handle_encoding;
11    range_del_block_handle.EncodeTo(&handle_encoding);
12    meta_index_block.Add(key, handle_encoding);
13 }
```

The metaindex key "leveldb.range_deletions" is analogous to "filter.Name" and allows `Table::ReadMeta` to locate the block on file open.

TableCache Changes (db/table_cache.h and db/table_cache.cc)

table_cache.h

`RangeDeletionList` is forward-declared and a new method is added:

Listing 49: db/table_cache.h – new method declaration

```
1 void GetRangeDeletions(uint64_t file_number, uint64_t file_size,
2                        RangeDeletionList* dest_list);
```

table_cache.cc – GetRangeDeletions

Listing 50: db/table_cache.cc – TableCache::GetRangeDeletions

```
1 void TableCache::GetRangeDeletions(uint64_t file_number,
2                                   uint64_t file_size,
3                                   RangeDeletionList* dest_list) {
4     Cache::Handle* handle = nullptr;
5     Status s = FindTable(file_number, file_size, &handle);
6     if (s.ok()) {
7         Table* table =
8             reinterpret_cast<TableAndFile*>(cache_>Value(handle))>table
9             ;
10        RangeDeletionList* table_dels = table->GetRangeDeletions();
11        if (table_dels != nullptr)
12            dest_list->MergeInto(table_dels);
13        cache_>Release(handle);
14    }
```

FindTable reuses the LRU table cache so the SSTable file is typically already open – no additional file I/O is required.

Version::Get Saver – db/version_set.cc

The **SaverState** enum inside the **Get** callback is extended with **kDeletedByRange**:

Listing 51: db/version_set.cc – extended SaverState

```
1 enum SaverState { kNotFound, kFound, kDeleted, kCorrupt,
   kDeletedByRange };
```

The dispatch inside the saver is extended to handle **kTypeRangeDeletion** marker entries that appear in the internal key stream (see above in Section 1, Read Path).

Compaction Integration – DoCompactionWork (db/db_impl.cc)

Collecting Input Range Tombstones

At the start of **DoCompactionWork**, tombstones are gathered from all input files:

Listing 52: db/db_impl.cc – collecting tombstones for compaction

```
1 RangeDeletionList compaction_range_dels;
2 for (int which = 0; which < 2; which++) {
3     for (int i = 0; i < compact->compaction->num_input_files(which); i
4         ++){
5         FileMetaData* f = compact->compaction->input(which, i);
6         table_cache_->GetRangeDeletions(f->number, f->file_size,
7             &compaction_range_dels);
8     }
9 }
```

Sorting and Deduplicating Tombstones

The collected tombstones are sorted by start key (ascending) with sequence number descending as a tie-breaker, then deduplicated, and then stored in a local **dels** vector copy:

Listing 53: db/db_impl.cc – tombstone sort and dedup

```
1 auto dels = compaction_range_dels.GetDeletions(); // by-value copy
2 if (!dels.empty()) {
3     std::sort(dels.begin(), dels.end(),
4         [this](const RangeDeletion& a, const RangeDeletion& b) {
5         int cmp = user_comparator()->Compare(a.start_key, b.start_key);
6         if (cmp != 0) return cmp < 0;
7         return a.seq > b.seq; // descending sequence
8     });
9     auto last = std::unique(dels.begin(), dels.end(),
10        [this](const RangeDeletion& a, const RangeDeletion& b) {
11        return user_comparator()->Compare(a.start_key, b.start_key) ==
12            0
13            && a.seq == b.seq;
14        });
15     dels.erase(last, dels.end());
16 }
```

```
16 size_t del_idx = 0;
```

Note that `GetDeletions()` returns a by-value copy (under its internal lock), so the compaction loop can safely iterate `dels` without holding any mutex or worrying about concurrent modifications.

Interleaving Tombstones with Point Keys

The compaction loop condition is extended to drain both point keys and pending range tombstones:

Listing 54: `db/db_impl.cc` – extended compaction loop condition

```
1 while ((input->Valid() || del_idx < dels.size()) &&
2        !shutting_down_.load(std::memory_order_acquire)) {
```

At each iteration, the loop decides whether the next entry is a tombstone or a point key by comparing `tombstone_ikey.Encode()` against `input->key()` using the internal comparator. If the tombstone sorts earlier, it is written first.

Dropping Point Keys Covered by Range Tombstones

When processing a point key, a range-deletion check is added before the existing superseded-version logic:

Listing 55: `db/db_impl.cc` – dropping covered point keys

```
1 if (compaction_range_dels.IsDeleted(ikey.user_key, ikey.sequence,
2                                     compact->smallest_snapshot,
3                                     user_comparator())) {
4     drop = true;
5 } else if (!has_current_user_key || ...) {
6     // existing first-occurrence logic
7 }
```

Output File Boundary Tracking

The original code set the smallest/largest keys of an output file unconditionally. With range tombstones potentially contributing to the output, the boundary tracking is changed to conditional comparisons:

Listing 56: `db/db_impl.cc` – conditional boundary tracking

```
1 if (compact->current_output()->smallest.empty() ||
2     user_comparator()->Compare(ExtractUserKey(key),
3                                compact->current_output()->smallest.user_key()) < 0) {
4     compact->current_output()->smallest.DecodeFrom(key);
5 }
6 if (compact->current_output()->largest.empty() ||
7     user_comparator()->Compare(ExtractUserKey(key),
8                                compact->current_output()->largest.user_key()) > 0) {
9     compact->current_output()->largest.DecodeFrom(key);
10 }
```

A special case is also handled: if a compaction produces *no* point keys but has pending range tombstones, a dedicated output file is opened and the tombstones are written into it (with their own start/end keys used as the file boundaries).

Propagating Active Tombstones Across Output Files

When a new output file is opened mid-compaction, any range tombstones whose ranges are still active (their `end_key` is beyond the current position) are re-written into the new file with an adjusted start key (using the previous file's largest key plus a null byte as the new start, clamped to the original tombstone's start key if larger). This ensures no point key in the new file is falsely not covered by the tombstone.

Memtable Flush Boundary Fix (db/db_impl.cc – WriteLevel0Table)

When flushing the Immutable MemTable to a Level-0 SSTable, the SSTable file's smallest/largest key bounds are computed from the data written by `BuildTable`. If the MemTable contains range tombstones, their start and end keys may extend beyond the point-key bounds. A fix expands the file's metadata bounds to include tombstone extents:

Listing 57: db/db_impl.cc – WriteLevel0Table boundary expansion

```

1  const RangeDeletionList* dels = mem->GetRangeDeletions();
2  if (dels != nullptr) {
3      for (const auto& del : dels->GetDeletions()) {
4          if (meta.smallest.empty() ||
5              user_comparator()->Compare(del.start_key,
6                                          meta.smallest.user_key()) < 0)
7              meta.smallest = InternalKey(del.start_key, del.seq,
8                                          kTypeRangeDeletion);
9          if (meta.largest.empty() ||
10             user_comparator()->Compare(del.end_key,
11                                         meta.largest.user_key()) > 0)
12             meta.largest = InternalKey(del.end_key, del.seq,
13                                         kTypeRangeDeletion);
14     }
15 }

```

db/write_batch_test.cc

The test printer in `PrintContents` is extended to recognise and print `kTypeRangeDeletion` entries:

Listing 58: db/write_batch_test.cc – kTypeRangeDeletion case

```

1  case kTypeRangeDeletion:
2      state.append("DeleteRange(");
3      state.append(ikey.user_key().ToString());
4      state.append(", ");
5      state.append(iter->value().ToString());
6      state.append(")");
7      count++;
8      break;

```

db/version_set.h – Friend and Accessor

DBImpl is added as a `friend` of `Version` to allow direct access to the `files_` array inside `DBImpl::GetRangeDeletions`. A new `files_at_level(int)` const accessor method is also added.

Interaction Between Range and Point Tombstones

- A range tombstone at sequence S_r deletes a value at sequence S_v if and only if $S_r > S_v$ – the tombstone is *newer*.
- A point tombstone at sequence S_p sets the `skipping` flag in `FindNextUserEntry`, suppressing all older versions of the same user key.
- A key written *after* a range tombstone ($S_v > S_r$) passes the `IsDeleted` check because `del.seq > found_seq` fails.

Summary of Data Flow

Stage	File	Key Function / Structure
User API	<code>include/leveldb/db.h</code>	<code>DB::DeleteRange</code>
Batch encoding	<code>db/write_batch.cc</code>	<code>WriteBatch::DeleteRange</code>
MemTable write	<code>db/write_batch.cc</code>	<code>MemTableInserter::DeleteRange</code>
MemTable storage	<code>db/memtable.cc</code>	<code>MemTable::Add</code>
In-memory index	<code>db/dbformat.cc</code>	<code>RangeDeletionList::Add</code> (mutex)
Deletion check	<code>db/dbformat.cc</code>	<code>RangeDeletionList::IsDeleted</code> (mutex)
SSTable write	<code>table/table_builder.cc</code>	<code>TableBuilder::Add, Finish</code>
SSTable read	<code>table/table.cc</code>	<code>Table::ReadRangeDeletions</code>
Cache layer	<code>db/table_cache.cc</code>	<code>TableCache::GetRangeDeletions</code>
Aggregation	<code>db/db_impl.cc</code>	<code>DBImpl::GetRangeDeletions</code>
Iterator enforce	<code>db/db_iter.cc</code>	<code>DBIter::FindNextUserEntry</code>
Point lookup	<code>table/table.cc</code>	<code>Table::InternalGet</code>
Version search	<code>db/version_set.cc</code>	<code>SaverState::kDeletedByRange</code>
Compaction drop	<code>db/db_impl.cc</code>	<code>DoCompactionWork</code>

Design and Implementation of the Manual Full Compaction Mechanism

Motivation and Overview

LevelDB's existing `CompactRange()` targets a key range on a single level. It does not drain the LSM tree completely or report statistics about the work done. The new `ForceFullCompaction()` facility drains *every* level sequentially into the level below, reducing the LSM tree to the minimum number of levels and physically reclaiming space occupied by deleted and range-deleted entries.

The mechanism is implemented in `db/db_impl.cc` and `db/db_impl.h` through:

- `ForceFullCompaction()` – the top-level orchestrator.
- `FlushedMemTable()` – synchronously flushes the active MemTable.
- `CompactWholeLevel(int level)` – drains a single level.
- `StatsForCompaction` – aggregates and prints statistics.

- `SingleCompactionRecord` – per-job I/O record.
- Two new `DBImpl` members: `force_full_compaction_in_progress_` and `records_forcecompaction_`.

New Data Structures (db/db_impl.h)

SingleCompactionRecord

Listing 59: db/db_impl.h – `SingleCompactionRecord`

```

1 struct SingleCompactionRecord {
2     SingleCompactionRecord()
3         : input_files(0), output_files(0), bytes_read(0), bytes_written
4           (0) {}
5     int     input_files;
6     int     output_files;
7     int64_t bytes_read;
8     int64_t bytes_written;
9 };

```

One record is filled per call to `DoCompactionWork` or per trivial-move or memtable-flush event while a `ForceFullCompaction` is running.

StatsForCompaction

Listing 60: db/db_impl.h – `StatsForCompaction` (defined in db_impl.cc)

```

1 struct StatsForCompaction {
2     int64_t num_compactions;
3     int64_t num_input_files;
4     int64_t num_output_files;
5     int64_t bytes_read;
6     int64_t bytes_written;
7     int64_t elapsed_micros;
8
9     void Print() const;           // writes to stdout
10    std::string FormatBytes(int64_t) const; // adaptive B/KiB/MiB/GiB
11 };

```

`FormatBytes` uses `<sstream>` and `<iomanip>` (both now included in db/db_impl.cc). `Print()` writes the human-readable report to `stdout` via `std::fwrite` and `std::fflush`.

New DBImpl Members

Listing 61: db/db_impl.h – new `DBImpl` members

```

1 bool force_full_compaction_in_progress_ GUARDED_BY(mutex_);
2 std::vector<SingleCompactionRecord>* records_forcecompaction_
3   GUARDED_BY(mutex_);

```

Both are initialised in the constructor:

Listing 62: db/db_impl.cc – constructor initialisation

```

1 force_full_compaction_in_progress_(false),
2 records_forcecompaction_(nullptr)

```


`records_forcecompaction_` acts as an opt-in channel: it is non-null only while `ForceFullCompaction` is executing, so the hot path of `DoCompactionWork` pays zero overhead during ordinary compactions.

State Flag: `force_full_compaction_in_progress_`

This Boolean guards several orthogonal behaviours:

Write Throttling in `Write()`

As described in Section 1, foreground writes are blocked with:

Listing 63: `db/db_impl.cc` – `Write()` throttle loop

```
1 while (updates != nullptr && ShouldWaitFullCompaction()) {  
2     background_work_finished_signal_.Wait();  
3 }  
4 Status usable_status = CheckDatabaseUsable();  
5 if (!usable_status.ok()) return usable_status;
```

Only real writes (`updates != nullptr`) are blocked. Internal null-batch writes used by `FlushedMemTable()` are deliberately exempt so the memtable flush can proceed during the compaction.

Throttling `Get()` and `NewIterator()`

`DBImpl::Get` and `DBImpl::NewIterator` are similarly extended to wait on `ShouldWaitFullCompaction()` before proceeding, ensuring reads see a stable post-compaction state and do not race with level-draining operations.

Scheduler Bypass in `MaybeScheduleCompaction()`

As shown earlier, the bypass suppresses routine auto-compaction scheduling while permitting immutable-memtable flushes and user-initiated manual compactions.

Helper Methods

Listing 64: `db/db_impl.cc` – `ShouldWaitFullCompaction` and `CheckDatabaseUsable`

```
1 bool DBImpl::ShouldWaitFullCompaction() const {  
2     const bool full_compaction_running =  
3         force_full_compaction_in_progress_;  
4     const bool database_healthy = bg_error_.ok();  
5     const bool database_running =  
6         !shutting_down_.load(std::memory_order_acquire);  
7     return full_compaction_running && database_healthy &&  
8         database_running;  
9 }  
10  
11 Status DBImpl::CheckDatabaseUsable() const {  
12     if (!bg_error_.ok())  
13         return bg_error_;  
14     if (shutting_down_.load(std::memory_order_acquire))  
15         return Status::IOError("Database is shutting down");  
16     return Status::OK();  
17 }
```

RecordForceCompactionStats

Listing 65: db/db_impl.cc – RecordForceCompactionStats

```

1 void DBImpl::RecordForceCompactionStats(int input_files, int
   output_files,
2                                     int64_t bytes_read,
3                                     int64_t bytes_written) {
4     mutex_.AssertHeld();
5     if (records_forcecompaction_ != nullptr) {
6         SingleCompactionRecord record_entry;
7         record_entry.input_files = input_files;
8         record_entry.output_files = output_files;
9         record_entry.bytes_read = bytes_read;
10        record_entry.bytes_written = bytes_written;
11        records_forcecompaction_ ->push_back(record_entry);
12    }
13 }

```

This is called from three sites under the mutex:

1. **WriteLevel0Table**: after a MemTable flush (0 input files, 1 output, 0 bytes read, file size bytes written).
2. **DoCompactionWork**: after **InstallCompactionResults** with actual input/output file counts and byte stats from **CompactionStats**.
3. The trivial-move path in **BackgroundCompaction**: 1 input, 1 output, file size for both read and written.

Destructor Change

The destructor waits for both **background_compaction_scheduled_** and **force_full_compaction_in_progress_** before proceeding with teardown:

Listing 66: db/db_impl.cc – destructor wait extended

```

1 while (background_compaction_scheduled_ ||
2        force_full_compaction_in_progress_) {
3     background_work_finished_signal_.Wait();
4 }
5 // was: while (background_compaction_scheduled_) { ... }

```

FlushedMemTable() (db/db_impl.cc)

Listing 67: db/db_impl.cc – FlushedMemTable

```

1 Status DBImpl::FlushedMemTable() {
2     WriteOptions options;
3     options.sync = true;
4     Status s = Write(options, nullptr); // triggers memtable rotation
5     if (!s.ok()) return s;
6     MutexLock l(&mutex_);
7     while (imm_ != nullptr && bg_error_.ok() &&
8            !shutting_down_.load(std::memory_order_acquire))
9         background_work_finished_signal_.Wait();
10    return CheckDatabaseUsable();
11 }

```

A null-batch `Write` with `sync=true` causes `MakeRoomForWrite` to rotate the active `MemTable` into the immutable position and schedule `CompactMemTable`. The function then waits until `imm_` is cleared, guaranteeing all acknowledged writes are durably on disk before inter-level compaction begins. This is called *before* the scheduler-bypass flag is active, so the background flush thread can proceed without deadlock.

CompactWholeLevel() (db/db_impl.cc)

Listing 68: db/db_impl.cc – CompactWholeLevel

```

1 Status DBImpl::CompactWholeLevel(int level) {
2     assert(level >= 0 && level + 1 < config::kNumLevels);
3
4     ManualCompaction manual;
5     manual.level = level;
6     manual.done = false;
7     manual.begin = nullptr;    // full key range
8     manual.end   = nullptr;
9     manual.tmp_storage.Clear();
10
11     MutexLock lock(&mutex_);
12     while (manual.done == false && bg_error_.ok() == true &&
13           shutting_down_.load(std::memory_order_acquire) == false) {
14         if (manual_compaction_ == nullptr) {
15             manual_compaction_ = &manual;
16             MaybeScheduleCompaction();
17         } else {
18             background_work_finished_signal_.Wait();
19         }
20     }
21     // Drain any residual scheduled background work
22     while (background_compaction_scheduled_)
23         background_work_finished_signal_.Wait();
24
25     if (manual_compaction_ == &manual)
26         manual_compaction_ = nullptr;
27     return CheckDatabaseUsable();
28 }

```

`manual.begin=nullptr` and `manual.end=nullptr` instruct the existing `VersionSet::CompactRange` to cover the entire key space. The outer loop terminates when `BackgroundCompaction` sets `manual.done` to `true` – which it does when `VersionSet::CompactRange` returns `nullptr`, indicating no more compaction work remains in the level. The second wait ensures the level is fully quiescent before the function returns.

Execution Flow of ForceFullCompaction()

Phase 1 – Acquire Control and Initialise Records

Listing 69: db/db_impl.cc – ForceFullCompaction phase 1

```

1 const uint64_t start_micros = env_>NowMicros();
2 Status status;
3 std::vector<SingleCompactionRecord> records;
4 {
5     MutexLock lock(&mutex_);

```

```

6 // If another ForceFullCompaction is running, wait for it to finish
7 while (force_full_compaction_in_progress_)
8     background_work_finished_signal_.Wait();
9 force_full_compaction_in_progress_ = true;
10 records_forcecompaction_ = &records;
11 }

```

The `records` vector lives on the stack; its address is stored in `records_forcecompaction_` so that `RecordForceCompactionStats` (called from within compaction callbacks under the mutex) can append to it. The guard against concurrent invocations ensures only one full compaction runs at a time.

Phase 2 – Flush the MemTable

`FlushedMemTable()` is called immediately after setting the flag. This step happens *before* the scheduler bypass is active: the flush is a null-batch write that must be processed by the normal write path and the normal background flush thread without any deadlock.

Phase 3 – Wait for Existing Background Work

After the memtable flush, the mutex is re-acquired to drain any in-progress background compaction or manual compaction:

Listing 70: `db/db_impl.cc` – `ForceFullCompaction` phase 3

```

1 {
2     MutexLock lock(&mutex_);
3     while ((background_compaction_scheduled_ == true ||
4            manual_compaction_ != nullptr) &&
5            bg_error_.ok() == true &&
6            shutting_down_.load(std::memory_order_acquire) == false)
7         background_work_finished_signal_.Wait();
8     status = CheckDatabaseUsable();
9 }

```

Phase 4 – Sequential Level Draining

Listing 71: `db/db_impl.cc` – `ForceFullCompaction` phase 4

```

1 // First, find the highest level that currently has files. We only
2 // need to
3 // compact up to this level. Pushing data further down into empty
4 // levels
5 // provides no benefit and causes unnecessary I/O.
6 int max_level_with_files = 1; // Vanilla LevelDB starts this at 1
7 {
8     MutexLock lock(&mutex_);
9     status = CheckDatabaseUsable();
10    if (!status.ok()) {
11        FinishFullCompaction();
12        return status;
13    }
14    // Match vanilla DBImpl::CompactRange by starting from level 1
15    for (int level = 1; level < config::kNumLevels; level++) {
16        if (versions_ -> NumLevelFiles(level) > 0) {
17            max_level_with_files = level;
18        }
19    }
20 }

```

```

16     }
17 }
18 }
19
20 // We loop strictly LESS THAN max_level_with_files, exactly mirroring
21 // vanilla DBImpl::CompactRange. This pulls all data down to merge
22 // with the
23 // deepest populated level, safely resolving tombstones without
24 // touching
25 // empty bottom levels.
26 for (int level = 0; level < max_level_with_files; level++) {
27     {
28         MutexLock lock(&mutex_);
29         status = CheckDatabaseUsable();
30         if (!status.ok()) {
31             break;
32         }
33         // If a level is already empty, no point in trying to compact it.
34         if (versions_ -> NumLevelFiles(level) == 0) {
35             continue;
36         }
37     }
38
39     // Trigger a compaction of the entire current level.
40     status = CompactWholeLevel(level);
41
42     if (!status.ok()) {
43         break;
44     }
45 }

```

Phase 5 – Cleanup and Broadcast (FinishFullCompaction lambda)

A cleanup lambda captures the records pointer and status by reference:

Listing 72: db/db_impl.cc – FinishFullCompaction cleanup lambda

```

1 auto FinishFullCompaction = [&]() {
2     MutexLock lock(&mutex_);
3     if (records_forcecompaction_ == &records)
4         records_forcecompaction_ = nullptr;
5     force_full_compaction_in_progress_ = false;
6     background_work_finished_signal_.SignalAll();
7     if (status.ok() && !bg_error_.ok())
8         status = bg_error_;
9 };

```

`SignalAll()` wakes all foreground writers that were blocked in `Write()`, all `Get()` callers blocked in `ShouldWaitFullCompaction()`, and any `NewIterator()` callers that were waiting. The cleanup lambda is called unconditionally (whether the compaction succeeded or encountered an error), so writers are never permanently stalled.

Phase 6 – Statistics Aggregation and Output

Listing 73: db/db_impl.cc – stats aggregation

```

1 const uint64_t end_micros = env_ -> NowMicros();

```

```

2 StatsForCompaction aggregated_stats;
3 aggregated_stats.elapsed_micros = static_cast<int64_t>(end_micros -
  start_micros);
4 aggregated_stats.num_compactions = static_cast<int64_t>(records.size
  ());
5 auto record = records.begin();
6 while (record != records.end()) {
7     aggregated_stats.num_input_files += record->input_files;
8     aggregated_stats.num_output_files += record->output_files;
9     aggregated_stats.bytes_read += record->bytes_read;
10    aggregated_stats.bytes_written += record->bytes_written;
11    ++record;
12 }
13 aggregated_stats.Print();

```

`StatsForCompaction::Print()` produces output of the form:

```

Number of compactions executed: 7
Number of input files: 12
Number of output files: 4
Total bytes read: 1.23 MiB
Total bytes written: 892.00 KiB
Elapsed time: 45231 microseconds

```

Byte counts are formatted adaptively (B / KiB / MiB / GiB) by `FormatBytes` using `std::fixed<<std::setprecision(2)` from `<sstream>` and `<iomanip>` (both added to `db/db_impl.cc`).

Write() Signal Broadcast

A `background_work_finished_signal_.SignalAll()` is added at the end of `Write()` after notifying the next head of the writer queue:

Listing 74: `db/db_impl.cc` – `Write()` signal at end

```

1 if (!writers_.empty())
2     writers_.front()->cv.Signal();
3 background_work_finished_signal_.SignalAll();    // NEW
4 return status;

```

This allows `ForceFullCompaction` (waiting in `ShouldWaitFullCompaction`) to be woken as soon as the preceding writes drain, reducing latency before the full compaction begins.

Interaction with the Background Compaction Thread

`ForceFullCompaction` does not bypass the background-thread machinery. Each `CompactWholeLevel` call installs a `ManualCompaction` descriptor and calls `MaybeScheduleCompaction()`, which posts `BGWork` to the thread pool via `env_->Schedule()`. The background thread enters `BackgroundCall` → `BackgroundCompaction` → `DoCompactionWork`, where all existing logic – SSTable merge-sort, range-deletion propagation, output file sizing, `InstallCompactionResults()`, `RemoveObsoleteFiles()` – is exercised unchanged.

After every compaction `BackgroundCall()` signals:

Listing 75: db/db_impl.cc – BackgroundCall signal

```

1 background_compaction_scheduled_ = false;
2 MaybeScheduleCompaction();
3 background_work_finished_signal_.SignalAll();

```

This wakes `CompactWholeLevel`'s waiting loop, allowing it to check `manual.done` and either schedule the next round or return.

Correctness Considerations

Crash safety. `FlushedMemTable` waits for `imm_` to be cleared before inter-level compaction begins; all acknowledged writes are durably on disk at that point. Every inter-level compaction itself writes and syncs output SSTables before committing a `VersionEdit` to the MANIFEST, preserving the normal crash invariants.

Snapshot isolation. `DoCompactionWork` reads the oldest live snapshot sequence number before the merge loop; entries whose sequence numbers are needed by any live snapshot are preserved exactly as in ordinary compactions.

Shutdown safety. Every inner loop checks `shutting_down_.load(std::memory_order_acquire)`. `FinishFullCompaction` resets `force_full_compaction_in_progress_` unconditionally, so writers are never permanently blocked if the database is destroyed mid-compaction.

Concurrent-invocation safety. The guard at the start of `ForceFullCompaction` waits for any in-progress invocation to finish before setting the flag, preventing two full compactions from running simultaneously and racing on `records_forcecompaction_`.

Deadlock avoidance in FlushedMemTable. `FlushedMemTable` calls `Write(options, nullptr)` which passes `updates == nullptr`, bypassing the `ShouldWaitFullCompaction` check. This is the deliberate design: the null-batch write must proceed even when `force_full_compaction_in_progress_` is true, otherwise `ForceFullCompaction` would deadlock waiting for a flush that can never start.

Evaluation of Implemented Features

This section evaluates the correctness and robustness of the three implemented features: `Scan(start_key, end_key)`, `DeleteRange(start_key, end_key)`, and `ForceFullCompaction()`.

The tests were executed.

The final test run executed 47 tests from 2 test suites. All selected tests passed successfully.

```

...
[      OK ] DBTest.RangeScanEvaluation (29 ms)
[      OK ] DBTest.DeleteRangeEvaluation (35 ms)

```

The purpose of the evaluation was not only to check that the APIs return `Status::OK()`, but also to verify boundary cases, visibility rules, interaction with tombstones, statistics reporting, and concurrent execution behavior.

RangeScan Evaluation

The first test group evaluates the correctness of the new `Scan(start_key, end_key)` API. The main requirement for this API is that it must return all live key-value pairs in the half-open interval $[start_key, end_key)$, while following the same visibility rules as `Get()`.

The following cases were tested:

- **Empty range scan:** Keys `a`, `b`, and `c` were inserted, and the range $[x, z)$ was scanned. Since no inserted key lies in this interval, the expected result was an empty result vector. The test passed.
- **Normal range scan:** Keys `a`, `b`, `c`, and `d` were inserted, and the range $[b, d)$ was scanned. The expected output was exactly the keys `b` and `c`, in sorted order. The scan returned these two keys correctly.
- **Half-open boundary check:** The range $[b, d)$ was used to verify that the start key is included and the end key is excluded. The result contained `b` but did not contain `d`, confirming the required half-open interval behavior.
- **Updated key visibility:** Key `b` was updated from its old value to `new_v2`. A scan over $[a, c)$ returned `a` and `b`, and the value of `b` was `new_v2`. This confirms that `Scan()` returns the latest visible version of a key.
- **Deleted key filtering:** After deleting key `b`, the same scan range $[a, c)$ returned only `a`. This verifies that point tombstones are respected during scans.
- **Snapshot visibility:** Key `s` was first inserted with value `old`. A snapshot was then created, after which `s` was updated to `new`. A scan using the snapshot over $[r, t)$ returned `s = old`. This confirms that range scans preserve snapshot isolation and do not incorrectly expose updates that happened after the snapshot was taken.

The result of this test group was:

```
[ RUN      ] DBTest.RangeScanEvaluation
[          OK ] DBTest.RangeScanEvaluation (29 ms)
```

Thus, the `Scan()` implementation correctly handles empty ranges, normal ranges, half-open boundaries, overwritten keys, deleted keys, and snapshot-based reads.

DeleteRange Evaluation

The second test group evaluates the new `DeleteRange(start_key, end_key)` API. The expected behavior is that all keys in the half-open interval $[start_key, end_key)$ are logically deleted, while keys outside the interval remain visible.

The following cases were tested:

- **Basic range deletion:** Keys `a`, `b`, `c`, and `d` were inserted. Then `DeleteRange([b, d))` was called. After the operation, `a` and `d` remained visible, while `b` and `c` returned `NOT_FOUND`. This verifies that the range deletion only affects keys inside the specified interval.
- **Half-open boundary behavior:** The same range $[b, d)$ was used to verify the interval boundary rule. Key `b` was deleted because the start key is included, while

key `d` remained visible because the end key is excluded.

- **Range delete followed by a new write:** After deleting the range `[b, d)`, key `c` was inserted again with value `new_v3`. A subsequent `Get(c)` returned `new_v3`. This confirms that a newer write after a range tombstone remains visible and is not incorrectly hidden by the older range deletion.
- **Write followed by range delete:** Key `e` was inserted and then the range `[d, f)` was deleted. Both `d` and `e` returned `NOT_FOUND`. This verifies that the range tombstone correctly hides older keys inside its interval.
- **Interaction with point deletion:** Key `g` was inserted and deleted using the existing point delete API. A range deletion covering `g` was then issued. The key still returned `NOT_FOUND`. This ensures that point deletions and range deletions do not interfere in a way that resurrects deleted data.
- **Correctness after full compaction:** Keys `h` and `i` were inserted, the range `[h, j)` was deleted, and then `ForceFullCompaction()` was invoked. After compaction, both `h` and `i` still returned `NOT_FOUND`. This confirms that range-deleted keys remain deleted after SSTables are compacted.

The result of this test group was:

```
[ RUN      ] DBTest.DeleteRangeEvaluation
Number of compactions executed: 1
Number of input files: 0
Number of output files: 1
Total bytes read: 0 B
Total bytes written: 339 B
Elapsed time: 11805 microseconds
[         OK ] DBTest.DeleteRangeEvaluation (35 ms)
```

This confirms that `DeleteRange()` implements the required half-open interval semantics, interacts correctly with point deletions and newer writes, and remains correct after full compaction.

Manual Full Compaction Evaluation

The third group of tests evaluates `ForceFullCompaction()`. These tests verify that full compaction works on empty databases, MemTable-only data, Level-0 data, multi-level data, deleted keys, overwritten keys, snapshots, concurrent foreground operations, and repeated compaction calls.

The following categories were tested:

- **Empty database:** `ForceFullCompaction()` was called on a newly created empty database. The function returned successfully and reported zero compactions, zero input files, zero output files, zero bytes read, and zero bytes written. This verifies that the no-work case is handled safely.
- **Repeated calls on an empty database:** The function was called multiple times on an empty database. All calls completed successfully, confirming that repeated no-op compactions do not create inconsistent state or deadlock.

- **Data only in MemTable:** A few keys were inserted without explicitly flushing the MemTable. After `ForceFullCompaction()`, all keys remained readable. This verifies that full compaction first flushes recent in-memory updates before processing on-disk levels.
- **Level-0 data:** Data was inserted and flushed so that files were created in Level 0. After full compaction, Level 0 became empty and the data was still readable. This confirms that Level-0 files are pushed into lower levels without data loss.
- **Data across multiple levels:** Larger datasets were inserted to create data spread across multiple levels. After full compaction, the tests verified that data remained correct and that Level 0 was empty.
- **Deleted key cleanup:** A key was inserted, deleted, and then full compaction was invoked. The key remained absent after compaction. This verifies that deletion markers are respected when compacting SSTables.
- **Overwritten key correctness:** A key was written multiple times with different values. After full compaction, the value returned by `Get()` was the latest value. This confirms that older obsolete versions are not exposed after compaction.
- **Snapshot preservation:** A snapshot was created before full compaction. A read using that snapshot after compaction still returned the value visible at the time of the snapshot. This confirms that full compaction does not violate snapshot visibility.
- **Concurrent writes during full compaction:** Writer threads were started while full compaction was in progress. The writes completed after full compaction finished, and the written values were readable. This verifies that foreground writes are safely coordinated with full compaction.
- **Concurrent reads during full compaction:** Read operations were also issued during full compaction. They completed safely and returned correct values. This verifies that foreground reads are also coordinated with full compaction.
- **Concurrent full compaction calls:** Multiple threads invoked `ForceFullCompaction()` concurrently. All calls returned successfully. This confirms that concurrent full compaction requests are serialized safely.
- **Manual compaction interaction:** A standard manual compaction and a full compaction were triggered close to each other. The full compaction waited safely while the manual compaction was active, and both operations completed successfully.
- **Deadlock safety:** Multiple write-compaction cycles were executed. The database remained usable after each cycle, confirming that the implementation does not deadlock under repeated use.
- **Level structure checks:** Several tests checked that Level 0 becomes empty after full compaction and that the total number of files does not grow unexpectedly. These tests confirm that compaction reorganizes the LSM tree rather than simply adding redundant files.
- **Iterator correctness after compaction:** An iterator was created after full compaction and used to verify that keys were still returned in sorted order. This confirms that compaction preserves LevelDB's ordering invariant.

- **Range tombstone edge cases:** Additional tests checked that range tombstones do not corrupt SSTable index boundaries, do not duplicate incorrectly across files, and do not cause valid keys outside the deleted range to disappear.

Representative results from the full compaction tests are shown below.

```
[ RUN      ] DBTest.ForceFullCompactionEvaluation
Number of compactions executed: 0
Number of input files: 0
Number of output files: 0
Total bytes read: 0 B
Total bytes written: 0 B
Elapsed time: 7859 microseconds
```

```
Number of compactions executed: 1
Number of input files: 0
Number of output files: 1
Total bytes read: 0 B
Total bytes written: 144 B
Elapsed time: 11970 microseconds
```

```
Number of compactions executed: 1
Number of input files: 0
Number of output files: 1
Total bytes read: 0 B
Total bytes written: 1.48 KiB
Elapsed time: 11267 microseconds
```

```
[          OK ] DBTest.ForceFullCompactionEvaluation (85 ms)
```

The following output shows a case where full compaction interacted with an existing manual compaction:

```
[ RUN      ] DBTest.ForceFullCompactionConcurrentManual
Number of compactions executed: 2
Number of input files: 4
Number of output files: 2
Total bytes read: 532 B
Total bytes written: 266 B
Elapsed time: 598569 microseconds
[          OK ] DBTest.ForceFullCompactionConcurrentManual (728 ms)
```

The following output shows a high-concurrency stress case:

```
[ RUN      ] FullCompactionTest.HighConcurrencyStress
Number of compactions executed: 1
Number of input files: 0
Number of output files: 1
Total bytes read: 0 B
Total bytes written: 200 B
Elapsed time: 13886 microseconds
```

```

Number of compactions executed: 3
Number of input files: 4
Number of output files: 3
Total bytes read: 54.78 KiB
Total bytes written: 51.04 KiB
Elapsed time: 69720 microseconds

```

```
[      OK ] FullCompactionTest.HighConcurrencyStress (2028 ms)
```

Therefore, `ForceFullCompaction()` correctly performs synchronous full compaction, prints the required statistics, preserves data correctness, and safely coordinates with foreground operations.

Explanation of Compaction Statistics Using a Simple Case

This subsection explains how the statistics printed by `ForceFullCompaction()` are obtained. Consider the following simple sequence of operations:

```

Put("k1", "v1")
Put("k2", "v2")
Put("k3", "v3")
Delete("k2")
ForceFullCompaction()

```

Before full compaction, the database contains three logical writes and one point deletion. In LevelDB, the delete operation does not immediately remove `k2` from old SSTables. Instead, it writes a tombstone. The actual cleanup happens during a true compaction (such as merging Level 0 into Level 1), when the compaction iterator sees both the tombstone and any older value for the same user key.

A simplified view of the data state is:

Key	Entry type	Meaning
k1	Value	Live key
k2	Deletion tombstone	Deletes older value of k2
k2	Old value	Obsolete because of tombstone
k3	Value	Live key

During a true compaction merge between levels, the system scans the input files in sorted internal-key order. For every key, it decides whether the entry should be kept or dropped.

- `k1` is a live value and has no newer deletion covering it, so it is written to the output SSTable.
- `k2` has a newer deletion tombstone. Once the compaction sees that the older value of `k2` is covered by this tombstone, the old value is dropped.
- If the tombstone for `k2` is no longer needed by any snapshot or lower level, the tombstone itself is also dropped. This physically removes the deleted key from the compacted output.
- `k3` is a live value and is written to the output SSTable.

After a true compaction, the simplified output becomes:

Key	Result after compaction
k1	Kept
k2	Dropped
k3	Kept

The printed statistics are then computed as follows:

- **Number of compactions executed:** This is incremented once for each compaction job performed during `ForceFullCompaction()`. If the database is empty and there is nothing to compact, this value is zero. If a MemTable flush creates one output file, or if one level is compacted into the next level, the count increases accordingly.
- **Number of input files:** This counts the SSTable files read as input by the compaction jobs. If the operation only flushes data from the MemTable to an SSTable, there are zero input SSTable files because the input was in memory, not an existing table file. This is why some test outputs show nonzero output files but zero input files.
- **Number of output files:** This counts the new SSTable files produced by the flush or compaction. In a small case, all surviving keys may fit into one output file. In larger cases, LevelDB may split the output into multiple SSTables.
- **Total bytes read:** This is the sum of the sizes of SSTable files read during compaction. If the full compaction only flushes the MemTable, the value can be 0 B, because no existing SSTable was read.
- **Total bytes written:** This is the total size of new SSTable files produced during compaction. This includes only the entries that survive compaction. Therefore, if obsolete values or tombstones are dropped, the written bytes may be smaller than the total logical data originally inserted.
- **Elapsed time:** This measures the wall-clock time taken by the full compaction operation, including flushing, waiting for active work, compacting levels, and printing statistics.

For example, one of the test outputs was:

```
Number of compactions executed: 1
Number of input files: 0
Number of output files: 1
Total bytes read: 0 B
Total bytes written: 155 B
Elapsed time: 11805 microseconds
```

This corresponds to a tiny case where the data was entirely in the MemTable. `ForceFullCompaction()` executed a single job to flush the MemTable directly to an SSTable. Because this was just a memory flush and not a true merge between levels, it did not read any existing SSTable input files, and the tombstones (like the one for **k2**) were actually **preserved** in the output. The system safely halted compaction here to avoid pushing the SSTable downward when there was no guaranteed meaningful overlap work left to do.

Another test output was:

Number of compactions executed: 2
 Number of input files: 4
 Number of output files: 2
 Total bytes read: 532 B
 Total bytes written: 266 B
 Elapsed time: 598569 microseconds

This represents a case where the database had already flushed data to Level 0, so `ForceFullCompaction()` actively merged Level 0 into Level 1. Because this was a true compaction reading existing SSTable files, it successfully evaluated the **k2** tombstone, realized no older data existed below it, and **dropped** the obsolete entries entirely. The total bytes written were smaller than the bytes read because of this cleanup.

Thus, the statistics printed by `ForceFullCompaction()` are consistent with the actual work performed by the storage engine: they reflect how many compaction jobs ran, how many files were consumed, how many files were produced, how much data was read, how much live data was written back, and how long the operation took.

Summary of Results

Table 1 summarizes the main evaluation results.

Feature	Main behavior tested	Result
<code>Scan()</code>	Empty ranges, normal ranges, half-open interval boundaries, overwritten keys, deleted keys, and snapshot visibility	Passed
<code>DeleteRange()</code>	Basic range deletion, boundary behavior, newer writes after range deletion, point deletion interaction, and correctness after full compaction	Passed
<code>ForceFullCompaction()</code>	Empty DB, MemTable flush, Level-0 draining, multi-level compaction, deleted key cleanup, overwritten key correctness, and statistics reporting	Passed
Concurrency behavior	Concurrent writes, concurrent reads, concurrent full compaction calls, manual compaction interaction, and stress testing	Passed
Range tombstone edge cases	Range tombstone persistence, SSTable boundary handling, index corruption prevention, and avoiding incorrect key disappearance	Passed

Table 1: Summary of evaluation results for the implemented LevelDB features.

Overall, the evaluation confirms that the implemented APIs behave correctly under normal usage, boundary cases, compaction-heavy workloads, and concurrent execution scenarios. All selected tests passed successfully.

Resources

To facilitate a deeper understanding of the LevelDB implementation done by us, the following link is recommended:

- **Project Repository:** `LevelDB_COP290`
This repository contains the implementation details and experimental setups used to analyze the LSM-Tree performance metrics.